

举例说明, 假设你需要在自己目录中编译一个源文件, `/home/dracula/src/the_sun_sucks.c`, 每一次对文件进行访问 (比如说, 首先要打开它, 然后要存储它, 还要进行编译等), VFS 都必须沿着嵌套的目录依次解析全部路径: `/`、`home`、`dracula`、`src` 和最终的 `the_sun_sucks.c`。为了避免每次访问该路径名都进行这种耗时的操作, VFS 会先在目录项缓存中搜索路径名, 如果找到了, 就无须花费那么大的力气了。相反, 如果该目录项在目录项缓存中并不存在, VFS 就必须自己通过遍历文件系统为每个路径分量解析路径, 解析完毕后, 再将目录项对象加入 `dcache` 中, 以便以后可以快速查找到它。

而 `dcache` 在一定意义上也提供对索引节点的缓存, 也就是 `icache`。和目录项对象相关的索引节点对象不会被释放, 因为目录项会让相关索引节点的使用计数为正, 这样就可以确保索引节点留在内存中。只要目录项被缓存, 其相应的索引节点也就被缓存了。所以像前面的例子, 只要路径名在缓存中找到了, 那么相应的索引节点肯定也在内存中缓存着。

因为文件访问呈现空间和时间的局部性, 所以对目录项和索引节点进行缓存非常有益。文件访问有时间上的局部性, 是因为程序可能会一次又一次地访问相同的文件。因此, 当一个文件被访问时, 所缓存的相关目录项和索引节点不久被命中的概率较高。文件访问具有空间的局部性是因为程序可能在同一个目录下访问多个文件, 因此一个文件对应的目录项缓存后极有可能被命中, 因为相关的文件可能在下次又被使用。

13.10 目录项操作

`dentry_operation` 结构体指明了 VFS 操作目录项的所有方法。

该结构定义在文件 `<linux/dcache.h>` 中。

```
struct dentry_operations {
    int (*d_revalidate) (struct dentry *, struct nameidata *);
    int (*d_hash) (struct dentry *, struct qstr *);
    int (*d_compare) (struct dentry *, struct qstr *, struct qstr *);
    int (*d_delete) (struct dentry *);
    void (*d_release) (struct dentry *);
    void (*d_iput) (struct dentry *, struct inode *);
    char *(*d_dname) (struct dentry *, char *, int);
};
```

下面给出函数的具体用法:

- `int d_revalidate(struct dentry *dentry, struct nameidata*);`

该函数判断目录对象是否有效。VFS 准备从 `dcache` 中使用一个目录项时, 会调用该函数。大部分文件系统将该方法置 `NULL`, 因为它们认为 `dcache` 中的目录项对象总是有效的。

- `int d_hash(struct dentry *dentry, struct qstr *name)`

该函数为目录项生成散列值, 当目录项需要加入到散列表中时, VFS 调用该函数。

- `int d_compare(struct dentry *dentry,`

```
struct qstr *name1,
struct qstr *name2)
```

VFS 调用该函数来比较 name1 和 name2 这两个文件名。多数文件系统使用 VFS 默认的操作，仅仅作字符串比较。对有些文件系统，比如 FAT，简单的字符串比较不能满足其需要。因为 FAT 文件系统不区分大小写，所以需要实现一种不区分大小写的字符串比较函数。注意使用该函数时需要加 dcache_lock 锁。

- int d_delete(struct dentry *dentry)

当目录项对象的 d_count 计数值等于 0 时，VFS 调用该函数。注意使用该函数需要加 dcache_lock 锁和目录项的 d_lock。

- void d_release(struct dentry *dentry)

当目录项对象将要被释放时，VFS 调用该函数，默认情况下，它什么也不做。

- void d_iput(struct dentry *dentry,
struct inode *inode)

当一个目录项对象丢失了其相关的索引节点时（也就是说磁盘索引节点被删除了），VFS 调用该函数。默认情况下 VFS 会调用 iput() 函数释放索引节点。如果文件系统重载了该函数，那么除了执行此文件系统特殊的工作外，还必须调用 iput() 函数。

13.11 文件对象

VFS 的最后一个主要对象是文件对象。文件对象表示进程已打开的文件。如果我们站在用户角度来看待 VFS，文件对象会首先进入我们的视野。进程直接处理的是文件，而不是超级块、索引节点或目录项。所以不必奇怪：文件对象包含我们非常熟悉的信息（如访问模式，当前偏移等），同样道理，文件操作和我们非常熟悉的系统调用 read() 和 write() 等也很类似。

文件对象是已打开的文件在内存中的表示。该对象（不是物理文件）由相应的 open() 系统调用创建，由 close() 系统调用撤销，所有这些文件相关的调用实际上都是文件操作表中定义的方法。因为多个进程可以同时打开和操作同一个文件，所以同一个文件也可能存在多个对应的文件对象。文件对象仅仅在进程观点上代表已打开文件，它反过来指向目录项对象（反过来指向索引节点），其实只有目录项对象才表示已打开的实际文件。虽然一个文件对应的文件对象不是唯一的，但对应的索引节点和目录项对象无疑是唯一的。

文件对象由 file 结构体表示，定义在文件 <linux/fs.h> 中，下面给出该结构体和各项的描述。

```
struct file {
    union {
        struct list_head    fu_list;        /* 文件对象链表 */
        struct rcu_head      fu_rcuhead;    /* 释放之后的 RCU 链表 */
    } f_u;
    struct path              f_path;        /* 包含目录项 */
    struct file_operations   *f_op;        /* 文件操作表 */
    spinlock_t               f_lock;        /* 单个文件结构锁 */
    atomic_t                 f_count;       /* 文件对象的使用计数 */
}
```



```

                                unsigned long,
                                unsigned long,
                                unsigned long);

int (*check_flags) (int);
int (*flock) (struct file *, int, struct file_lock *);
ssize_t (*splice_write) (struct pipe_inode_info *,
                        struct file *,
                        loff_t *,
                        size_t,
                        unsigned int);
ssize_t (*splice_read) (struct file *,
                        loff_t *,
                        struct pipe_inode_info *,
                        size_t,
                        unsigned int);
int (*setlease) (struct file *, long, struct file_lock **);
};

```

具体的文件系统可以为每一种操作做专门的实现，或者如果存在通用操作，也可以使用通用操作。一般在基于 Unix 的文件系统上，这些通用操作效果都不错。并不要求实际文件系统实现文件操作函数表中的所有方法——虽然不实现最基础的那些操作显然是很不明智的，对不感兴趣的操作完全可以简单地将该函数指针置为 NULL。

下面给出操作的用法说明：

- `loff_t lseek(struct file *file, loff_t offset, int origin)`

该函数用于更新偏移量指针，由系统调用 `lseek()` 调用它。

- `ssize_t read(struct file *file, char *buf, size_t count, loff_t *offset)`

该函数从给定文件的 `offset` 偏移处读取 `count` 字节的数据到 `buf` 中，同时更新文件指针。由系统调用 `read()` 调用它。

- `ssize_t aio_read(struct kiocb *iocb, char *buf, size_t count, loff_t offset)`

该函数从 `iocb` 描述的文件里，以同步方式读取 `count` 字节的数据到 `buf` 中。由系统调用 `aio_read()` 调用它。

- `ssize_t write(struct file *file, const char *buf, size_t count, loff_t *offset)`

该函数从给定的 `buf` 中取出 `count` 字节的数据，写入给定文件的 `offset` 偏移处，同时更新文件指针。由系统调用 `write()` 调用它。

- `ssize_t aio_write(struct kiocb *iocb,
 const char *buf,
 size_t count, loff_t offset)`

该函数以同步方式从给定的 buf 中取出 count 字节的数据，写入由 iocb 描述的文件中。由系统调用 `aio_write()` 调用它。

- `int readdir(struct file *file, void *dirent, filldir_t filldir)`

该函数返回目录列表中的下一个目录。由系统调用 `readdir()` 调用它。

- `unsigned int poll(struct file *file,
 struct poll_table_struct *poll_table)`

该函数睡眠等待给定文件活动。由系统调用 `poll()` 调用它。

- `int ioctl(struct inode *inode,
 struct file *file,
 unsigned int cmd,
 unsigned long arg)`

该函数用来给设备发送命令参数对。当文件是一个被打开的设备节点时，可以通过它进行设置操作。由系统调用 `ioctl()` 调用它。调用者必须持有 BKL。

- `int unlocked_ioctl(struct file *file,
 unsigned int cmd,
 unsigned long arg)`

其实现与 `ioctl()` 有类似的功能，只不过不需要调用者持有 BKL。如果用户空间调用 `ioctl()` 系统调用，VFS 便可以调用 `unlocked_ioctl()`（凡是 `ioctl()` 出现的场所）。因此文件系统只需要实现其中的一个，一般优先实现 `unlocked_ioctl()`。

- `int compat_ioctl(struct file *file,
 unsigned int cmd,
 unsigned long arg)`

该函数是 `ioctl()` 函数的可移植变种，被 32 位应用程序用在 64 位系统上。这个函数被设计成即使在 64 位的体系结构上对 32 位也是安全的，它可以进行必要的字大小转换。新的驱动程序应该设计自己的 `ioctl` 命令以便所有的驱动程序都是可移植的，从而使得 `compat_ioctl()` 和 `unlocked_ioctl()` 指向同一个函数。像 `compat_ioctl()` 和 `unlocked_ioctl()` 一样都不必持有 BKL。

- `int mmap(struct file *file, struct vm_area_struct *vma)`

该函数将给定的文件映射到指定的地址空间上。由系统调用 `mmap()` 调用它。

- `int open(struct inode *inode,
 struct file *file)`

该函数创建一个新的文件对象，并将它和相应的索引节点对象关联起来。由系统调用 `open()` 调用它。

- `int flush(struct file *file)`

当已打开文件的引用计数减少时，该函数被 VFS 调用。它的作用根据具体文件系统而定。

- `int release(struct inode *inode,
 struct file *file)`

当文件的最后一个引用被注销时（比如，当最后一个共享文件描述符的进程调用了 `close()` 或退出时），该函数会被 VFS 调用。它的作用根据具体文件系统而定。

- `int fsync(struct file *file,
 struct dentry *dentry,
 int datasync)`

将给定文件的所有被缓存数据写回磁盘。由系统调用 `fsync()` 调用它。

- `int aio_fsync(struct kiocb *iocb,
 int datasync)`

将 `iocb` 描述的文件的所有被缓存数据写回到磁盘。由系统调用 `aio_fsync()` 调用它。

- `int fasync(int fd, struct file *file, int on)`

该函数用于打开或关闭异步 I/O 的通告信号。

- `int lock(struct file *file, int cmd, struct file_lock *lock)`

该函数用于给指定文件上锁。

- `ssize_t readv(struct file *file,
 const struct iovec *vector,
 unsigned long count,
 loff_t *offset)`

该函数从给定文件中读取数据，并将其写入由 `vector` 描述的 `count` 个缓冲中去，同时增加文件的偏移量。由系统调用 `readv()` 调用它。

- `ssize_t writev(struct file *file,
 const struct iovec *vector,
 unsigned long count,
 loff_t *offset)`

该函数将由 `vector` 描述的 `count` 个缓冲中的数据写入到由 `file` 指定的文件中去，同时减小文件的偏移量。由系统调用 `writev()` 调用它。

- `ssize_t sendfile(struct file *file,
 loff_t *offset,
 size_t size,
 read_actor_t actor,
 void *target)`

该函数用于从一个文件拷贝数据到另一个文件中，它执行的拷贝操作完全在内核中完成，避免了向用户空间进行不必要的拷贝。由系统调用 `sendfile()` 调用它。

- `ssize_t sendpage(struct file *file,
 struct page *page,
 int offset, size_t size,
 loff_t *pos, int more)`

该函数用来从一个文件向另一个文件发送数据。

- `unsigned long get_unmapped_area(struct file *file,
 unsigned long addr,
 unsigned long len,
 unsigned long offset,
 unsigned long flags)`

该函数用于获取未使用的地址空间来映射给定的文件。

- `int check_flags(int flags)`

当给出 SETFL 命令时，这个函数用来检查传递给 `fcntl()` 系统调用的 `flags` 的有效性。与大多数 VFS 操作一样，文件系统不必实现 `check_flags()`——目前，只有在 NFS 文件系统中实现了。这个函数能使文件系统限制无效的 SETFL 标志，不进行限制的话，普通的 `fcntl()` 函数能使标志生效。在 NFS 文件系统中，不允许把 `O_APPEND` 和 `O_DIRECT` 相结合。

- `int flock(struct file *filp,
 int cmd,
 struct file_lock *fl)`

这个函数用来实现 `flock()` 系统调用，该调用提供忠告锁。

如此之多的 `ioctl`s

不久之前，只有一个单独的 `ioctl` 方法。如今，有三个相关的方法。`unlocked_ioctl()` 和 `ioctl` 相同，不过前者在无大内核锁（BKL）情况下被调用。因此函数的作者必须确保适当的同步。因为大内核锁是粗粒度、低效的锁，驱动程序应当实现 `unlocked_ioctl()` 而不是 `ioctl()`。

`compat_ioctl()` 也在无大内核锁的情况下被调用，但是它的目的是为 64 位的系统提供 32 位 `ioctl` 的兼容方法。至于你如何实现它取决于现有的 `ioctl` 命令。早期的驱动程序隐含有确定大小的类型（如 `long`），应该实现适用于 32 位应用的 `compat_ioctl()` 方法。这通常意味着把 32 位值转换为 64 位内核中合适的类型。新驱动程序重新设计 `ioctl` 命令，应该确保所有的参数和数据都有明确大小的数据类型，在 32 位系统上运行 32 位应用是安全的，在 64 位系统上运行 32 位应用也是安全的，在 64 位系统上运行 64 位应用更是安全的。这些驱动程序可以让 `compat_ioctl()` 函数指针和 `unlocked_ioctl()` 函数指针指向同一函数。

13.13 和文件系统相关的数据结构

除了以上几种 VFS 基础对象外，内核还使用了另外一些标准数据结构来管理文件系统的其他相关数据。第一个对象是 `file_system_type`，用来描述各种特定文件系统类型，比如 `ext3`、`ext4` 或 `UDF`。第二个结构体是 `vfsmount`，用来描述一个安装文件系统的实例。

因为 Linux 支持众多不同的文件系统，所以内核必须由一个特殊的结构来描述每种文件系统的功能和行为。`file_system_type` 结构体被定义在 `<linux/fs.h>` 中，具体实现如下：

```
struct file_system_type {
    const char      *name;          /* 文件系统的名字 */
    int             fs_flags;       /* 文件系统类型标志 */
    .
```

```

/* 下面的函数用来从磁盘中读取超级块 */
struct super_block      *(*get_sb)      (struct file_system_type *, int,
                                          char *, void *);

/* 下面的函数用来终止访问超级块 */
void                    (*kill_sb)      (struct super_block *);

struct module           *owner;         /* 文件系统模块 */
struct file_system_type *next;         /* 链表中下一个文件系统类型 */
struct list_head        fs_supers;     /* 超级块对象链表 */

/* 剩下的几个字段运行时使锁生效 */
struct lock_class_key   s_lock_key;
struct lock_class_key   s_umount_key;
struct lock_class_key   i_lock_key;
struct lock_class_key   i_mutex_key;
struct lock_class_key   i_mutex_dir_key;
struct lock_class_key   i_alloc_sem_key;
};

```

get_sb() 函数从磁盘上读取超级块，并且在文件系统被安装时，在内存中组装超级块对象。剩余的函数描述文件系统的属性。

每种文件系统，不管有多少个实例安装到系统中，还是根本就没有安装到系统中，都只有一个 file_system_type 结构。

更有趣的事情是，当文件系统被实际安装时，将有一个 vfsmount 结构体在安装点被创建。该结构体用来代表文件系统的实例——换句话说，代表一个安装点。

vfsmount 结构被定义在 <linux/mount.h> 中，下面是具体结构：

```

struct vfsmount {
    struct list_head    mnt_hash;           /* 散列表 */
    struct vfsmount     *mnt_parent;        /* 父文件系统 */
    struct dentry       *mnt_mountpoint;    /* 安装点的目录项 */
    struct dentry       *mnt_root;         /* 该文件系统的根目录项 */
    struct super_block   *mnt_sb;          /* 该文件系统的超级块 */
    struct list_head    mnt_mounts;        /* 子文件系统链表 */
    struct list_head    mnt_child;         /* 子文件系统链表 */
    int                 mnt_flags;         /* 安装标志 */
    char                *mnt_devname;      /* 设备文件名 */
    struct list_head    mnt_list;          /* 描述符链表 */
    struct list_head    mnt_expire;        /* 在到期链表中的入口 */
    struct list_head    mnt_share;         /* 在共享安装链表中的入口 */
    struct list_head    mnt_slave_list;    /* 从安装链表 */
    struct list_head    mnt_slave;        /* 从安装链表中的入口 */
    struct vfsmount     *mnt_master;       /* 从安装链表的主人 */
    struct mnt_namespace *mnt_namespace;   /* 相关的命名空间 */
    int                 mnt_id;            /* 安装标识符 */
    int                 mnt_group_id;      /* 组标识符 */
    atomic_t            mnt_count;         /* 使用计数 */
    int                 mnt_expiry_mark;   /* 如果标记为到期，则值为真 */
    int                 mnt_pinned;        /* “钉住”进程计数 */
    int                 mnt_ghosts;       /* “镜像”引用计数 */
    atomic_t            __mnt_writers;     /* 写者引用计数 */
};

```

理清文件系统和所有其他安装点间的关系，是维护所有安装点链表中最复杂的工作。所以

vfsmount 结构体中维护的各种链表就是为了能够跟踪这些关联信息。

vfsmount 结构还保存了在安装时指定的标志信息，该信息存储在 mnt_flags 域中。表 13-1 列出了标准的安装标志。

表 13-1 标准安装标志列表

标 志	描 述
MNT_NOSUID	禁止该文件系统的可执行文件设置 setuid 和 setgid 标志
MNT_MODEV	禁止访问该文件系统上的设备文件
MNT_NOEXEC	禁止执行该文件系统上的可执行文件

安装那些管理员不充分信任的移动设备时，这些标志很有用处。它们和其他一些很少用的标志一起定义在 <linux/mount.h> 中。

13.14 和进程相关的数据结构

系统中的每一个进程都有自己的一组打开的文件，像根文件系统、当前工作目录、安装点等。有三个数据结构将 VFS 层和系统的进程紧密联系在一起，它们分别是：file_struct、fs_struct 和 namespace 结构体。

file_struct 结构体定义在文件 <linux/fdtable.h> 中。该结构体由进程描述符中的 files 目录项指向。所有与单个进程（per-process）相关的信息（如打开的文件及文件描述符）都包含在其中，其结构和描述如下：

```
struct files_struct {
    atomic_t          count;           /* 结构的使用计数 */
    struct fdtable     *fdt;           /* 指向其他 fd 表的指针 */
    struct fdtable     fdtab;          /* 基 fd 表 */
    spinlock_t        file_lock;       /* 单个文件的锁 */
    int                next_fd;         /* 缓存下一个可用的 fd */
    struct embedded_fd_set close_on_exec_init; /* exec() 时关闭的文件描述符链表 */
    struct embedded_fd_set open_fds_init; /* 打开的文件描述符链表 */
    struct file        *fd_array[NR_OPEN_DEFAULT]; /* 缺省的文件对象数组 */
};
```

fd_array 数组指针指向已打开的文件对象。因为 NR_OPEN_DEFAULT 等于 BITS_PER_LONG，在 64 位机器体系结构中这个宏的值为 64，所以该数组可以容纳 64 个文件对象。如果一个进程所打开的文件对象超过 64 个，内核将分配一个新数组，并且将 fdt 指针指向它。所以对适当数量的文件对象的访问会执行得很快，因为它是对静态数组进行的操作；如果一个进程打开的文件数量过多，那么内核就需要建立新数组。所以如果系统中有大量的进程都要打开超过 64 个文件，为了优化性能，管理员可以适当增大 NR_OPEN_DEFAULT 的预定义值。

和进程相关的第二个结构体是 fs_struct。该结构由进程描述符的 fs 域指向。它包含文件系统和进程相关的信息，定义在文件 <linux/fs_struct.h> 中，下面是它的具体结构体和各项描述：

```
struct fs_struct {
    int                users;           /* 用户数目 */
    rwlock_t          lock;            /* 保护该结构体的锁 */
};
```

```

int          umask;          /* 掩码 */
int          in_exec;        /* 当前正在执行的文件 */
struct path root;           /* 根目录路径 */
struct path pwd;            /* 当前工作目录的路径 */
};

```

该结构包含了当前进程的当前工作目录 (pwd) 和根目录。

第三个也是最后一个相关结构体是 namespace 结构体。它定义在文件 <linux/mmt_namespace.h> 中, 由进程描述符中的 mmt_namespace 域指向。2.4 版内核以后, 单进程命名空间被加入到内核中, 它使得每一个进程在系统中都看到唯一的安装文件系统——不仅是唯一的根目录, 而且是唯一的文件系统层次结构。下面是其具体结构和描述:

```

struct mmt_namespace {
    atomic_t          count; /* 结构的使用计数 */
    struct vfsmount    *root; /* 根目录的安装点对象 */
    struct list_head   list;  /* 安装点链表 */
    wait_queue_head_t poll;   /* 轮询的等待队列 */
    int               event;  /* 事件计数 */
};

```

list 域是连接已安装文件系统的双向链表, 它包含的元素组成了全体命名空间。

上述这些数据结构都是通过进程描述符连接起来的。对多数进程来说, 它们的描述符都指向唯一的 files_struct 和 fs_struct 结构体。但是, 对于那些使用克隆标志 CLONE_FILES 或 CLONE_FS 创建的进程, 会共享^①这两个结构体。所以多个进程描述符可能指向同一个 files_struct 或 fs_struct 结构体。每个结构体都维护一个 count 域作为引用计数, 它防止在进程正使用该结构时, 该结构被撤销。

namespace 结构体的使用方法却和前两种结构体完全不同, 默认情况下, 所有的进程共享同样的命名空间 (也就是, 它们都从相同的挂载表中看到同一个文件系统层次结构)。只有在进行 clone() 操作时使用 CLONE_NEWNS 标志, 才会给进程一个唯一的命名空间结构体的拷贝。因为大多数进程不提供这个标志, 所有进程都继承其父进程的命名空间。因此, 在大多数系统上只有一个命名空间, 不过, CLONE_NEWNS 标志可以使这一功能失效。

13.15 小结

Linux 支持了相当多种类的文件系统。从本地文件系统 (如 ext3 和 ext4) 到网络文件系统 (如 NFS 和 Coda), Linux 在标准内核中已支持的文件系统超过 60 种。VFS 层提供给这些不同文件系统一个统一的实现框架, 而且也提供了能和标准系统调用交互工作的统一接口。由于 VFS 层的存在, 使得在 Linux 上实现新文件系统的工作变得简单起来, 它可以轻松地使这些文件系统通过标准 Unix 系统调用而协同工作。

本章描述了 VFS 的目的, 讨论了各种数据结构, 包括最重要的索引节点、目录项以及超级块对象。第 14 章将讨论数据如何从物理上存放在文件系统中。

① 线程通常在创建时使用 CLONE_FILES 和 CLONE_FS 标志, 所以多个线程共享一个 file_struct 结构体和 fs_struct 结构体。但另一方面, 普通进程没有指定这些标志, 所以它们有自己的文件系统信息和打开文件表。

第 14 章

块 I/O 层

系统中能够随机（不需要按顺序）访问固定大小数据片（chunks）的硬件设备称作块设备，这些固定大小的数据片就称作块。最常见的块设备是硬盘，除此以外，还有软盘驱动器、蓝光光驱和闪存等许多其他块设备。注意，它们都是以安装文件系统的方式使用的——这也是块设备一般的访问方式。

另一种基本的设备类型是字符设备。字符设备按照字符流的方式被有序访问，像串口和键盘就属于字符设备。如果一个硬件设备是以字符流的方式被访问的话，那就应该将它归于字符设备；反过来，如果一个设备是随机（无序的）访问的，那么它就属于块设备。

对于这两种类型的设备，它们的区别在于是否可以随机访问数据——换句话说，就是能否在访问设备时随意地从一个位置跳转到另一个位置。举个例子，键盘这种设备提供的就是一个数据流，当你输入“wolf”这个字符串时，键盘驱动程序会按照和输入完全相同的顺序返回这个由四个字符组成的数据流。如果让键盘驱动程序打乱顺序来读字符串，或读取其他字符，都是没有意义的。所以键盘就是一种典型的字符设备，它提供的就是用户从键盘输入的字符流。对键盘进行读操作会得到一个字符流，首先是“w”，然后是“o”，再是“l”，最后是“x”。当没人敲键盘时，字符流就是空的。硬盘设备的情况就不大一样了。硬盘设备的驱动可能要求读取磁盘上任意块的内容，然后又转去读取别的块的内容，而被读取的块在磁盘上位置不一定要连续。所以说硬盘的数据可以被随机访问，而不是以流的方式被访问，因此它是一个块设备。

内核管理块设备要比管理字符设备细致得多，需要考虑的问题和完成的工作相对于字符设备来说要复杂许多。这是因为字符设备仅仅需要控制一个位置——当前位置，而块设备访问的位置必须能够在介质的不同区间前后移动。所以事实上内核不必提供一个专门的子系统来管理字符设备，但是对块设备的管理却必须要有一个专门的提供服务的子系统。不仅仅是因为块设备的复杂性远远高于字符设备，更重要的原因是块设备对执行性能的要求很高；对硬盘每多一份利用都会对整个系统的性能带来提升，其效果要远远比键盘吞吐速度成倍地提高大得多。另外，我们将会看到，块设备的复杂性会为这种优化留下很大的施展空间。这一章的主题就是讨论内核如何对块设备和块设备的请求进行管理。该部分在内核中称作块 I/O 层。有趣的是，改写块 I/O 层正是 2.5 开发版内核的主要目标。本章涵盖了 2.6 版内核中所有新的块 I/O 层。

14.1 剖析一个块设备

块设备中最小的可寻址单元是扇区。扇区大小一般是 2 的整数倍，而最常见的是 512 字节。扇区的大小是设备的物理属性，扇区是所有块设备的基本单元——块设备无法对比它还小的单元进行寻址和操作，尽管许多块设备能够一次对多个扇区进行操作。虽然大多数块设备的扇区大小

都是 512 字节，不过其他大小的扇区也很常见。比如，很多 CD-ROM 盘的扇区都是 2KB 大小。

因为各种软件的用途不同，所以它们都会用到自己的最小逻辑可寻址单元——块。块是文件系统的一种抽象——只能基于块来访问文件系统。虽然物理磁盘寻址是按照扇区级进行的，但是内核执行的所有磁盘操作都是按照块进行的。由于扇区是设备的最小可寻址单元，所以块不能比扇区还小，只能数倍于扇区大小。另外，内核（对有扇区的硬件设备）还要求块大小是 2 的整数倍，而且不能超过一个页的长度（请看第 12 章和第 19 章）^①。所以，对块大小的最终要求是，必须是扇区大小的 2 的整数倍，并且要小于页面大小。所以通常块大小是 512 字节、1KB 或 4KB。

扇区和块还有一些不同的叫法，为了不引起混淆，我们在这里简要介绍一下它们的其他名称。扇区——设备的最小寻址单元，有时会称作“硬扇区”或“设备块”；同样的，块——文件系统的最小寻址单元，有时会称作“文件块”或“I/O 块”。在这一章里，会一直使用“扇区”和“块”这两个术语，但你还是应该记住它们的这些别名。图 14-1 是扇区和缓冲区之间的关系图。

至少相对于硬盘而言，另外一些术语更通用——如簇、柱面以及磁头。这些表示是针对某些特定的块设备的，大多数情况下，对用户空间的软件是不可见的。扇区这一术语之所以对内核重要，是因为所有设备的 I/O 必须以扇区为单位进行操作。以此类推，内核所使用的“块”这一高级概念就是建立在扇区之上的。

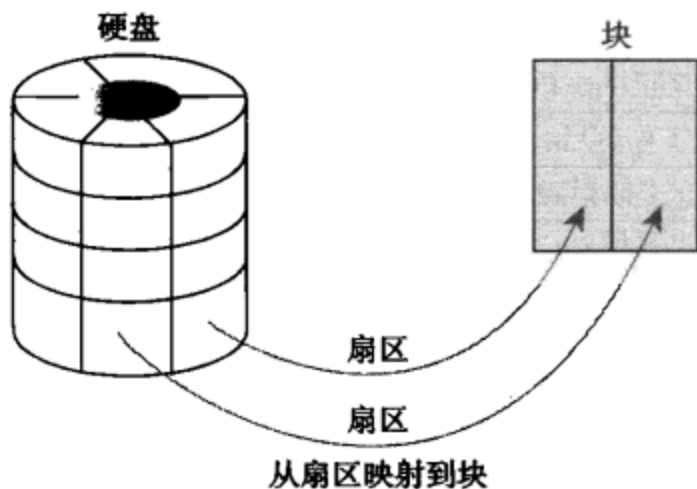


图 14-1 扇区和缓冲区之间的关系

14.2 缓冲区和缓冲区头

当一个块被调入内存时（也就是说，在读入后或等待写出时），它要存储在一个缓冲区中。每个缓冲区与一个块对应，它相当于是磁盘块在内存中的表示。前面提到过，块包含一个或多个扇区，但大小不能超过一个页面，所以一个页可以容纳一个或多个内存中的块。由于内核在处理数据时需要一些相关的控制信息（比如块属于哪一个块设备，块对应于哪个缓冲区等），所以每一个缓冲区都有一个对应的描述符。该描述符用 `buffer_head` 结构体表示，称作缓冲区头，在文件 `<linux/buffer_head.h>` 中定义，它包含了内核操作缓冲区所需要的全部信息。

^① 这个认为的限制可能会遗留到以后，但是强制块的大小等于或小于页大小无疑简化了内核。

下面给出缓冲区头结构体和其中各个域的说明：

```
struct buffer_head {
    unsigned long b_state;           /* 缓冲区状态标志 */
    struct buffer_head *b_this_page; /* 页面中的缓冲区 */
    struct page *b_page;             /* 存储缓冲区的页面 */
    sector_t b_blocknr;              /* 起始块号 */
    size_t b_size;                   /* 映像的大小 */
    char *b_data;                    /* 页面内的数据指针 */
    struct block_device *b_bdev;      /* 相关联的块设备 */
    bh_end_io_t *b_end_io;            /* I/O 完成方法 */
    void *b_private;                  /* io 完成方法 */
    struct list_head b_assoc_buffers; /* 相关的映射链表 */
    struct address_space *b_assoc_map; /* 相关的地址空间 */
    atomic_t b_count;                 /* 缓冲区使用计数 */
};
```

b_state 域表示缓冲区的状态，可以是表 14-1 中一种标志或多种标志的组合。合法的标志存放在 bh_state_bits 枚举中，该枚举在 <linux/buffer_head.h> 中定义。

表 14-1 bh_state 标志

状态标志	意 义
BH_Uptodate	该缓冲区包含可用数据
BH_Dirty	该缓冲区是脏的（缓存中的内容比磁盘中的块内容新，所以缓冲区内容必须被写回磁盘）
BH_Lock	该缓冲区正在被 I/O 操作使用，被锁定以防被并发访问
BH_Req	该缓冲区有 I/O 请求操作
BH_Mapped	该缓冲区是映射磁盘块的可用缓冲区
BH_New	缓冲区是通过 get_block() 刚刚映射的，尚且不能访问
BH_Async_Read	该缓冲区正通过 end_buffer_async_read() 被异步 I/O 读操作使用
BH_Async_write	该缓冲区正通过 end_buffer_async_write() 被异步 I/O 写操作使用
BH_Delay	该缓冲区尚未和磁盘块关联
BH_Boundary	该缓冲区处于连续块区的边界——下一个块不再连续
BH_Write_EIO	该缓冲区在写的时候遇到 I/O 错误
BH_Ordered	顺序写
BH_Eopnotsupp	该缓冲区发生“不被支持”错误
BH_Unwritten	该缓冲区在硬盘上的空间已被申请但是没有实际的数据写出
BH_Quiet	此缓冲区禁止错误

bh_state_bits 列表还包含了一个特殊标志——BH_PrivateStart，该标志不是可用状态标志，使用它是为了指明可被其他代码使用的起始位。块 I/O 层不会使用 BH_PrivateStart 或更高的位。那么某个驱动程序希望通过 b_state 域存储信息时就可以安全地使用这些位。驱动程序可以在这些位中定义自己的状态标志，只要保证自定义的状态标志不与块 I/O 层的专用位发生冲突就可以了。

b_count 域表示缓冲区的使用记数，可通过两个定义在文件 <linux/buffer_head.h> 中的内联函数对此域进行增减。


```
static inline void get_bh(struct buffer_head *bh)
{
    atomic_inc(&bh->b_count);
}

static inline void put_bh(struct buffer_head *bh)
{
    atomic_dec(&bh->b_count);
}
```

在操作缓冲区头之前，应该先使用 `get_bh()` 函数增加缓冲区头的引用计数，确保该缓冲区头不会再被分配出去；当完成对缓冲区头的操作之后，还必须使用 `put_bh()` 函数减少引用计数。

与缓冲区对应的磁盘物理块由 `b_blocknr-th` 域索引，该值是 `b_bdev` 域指明的块设备中的逻辑块号。

与缓冲区对应的内存物理页由 `b_page` 域表示，另外，`b_data` 域直接指向相应的块（它位于 `b_page` 域所指明的页面中的某个位置上），块的大小由 `b_size` 域表示，所以块在内存中的起始位置在 `b_data` 处，结束位置在 `(b_data + b_size)` 处。

缓冲区头的目的在于描述磁盘块和物理内存缓冲区（在特定页面上的字节序列）之间的映射关系。这个结构体在内核中只扮演一个描述符的角色，说明从缓冲区到块的映射关系。

在 2.6 内核以前，缓冲区头的作用比现在还要重要。因为缓冲区头作为内核中的 I/O 操作单元，不仅仅描述了从磁盘块到物理内存的映射，而且还是所有块 I/O 操作的容器。可是，将缓冲区头作为 I/O 操作单元带来了两个弊端。首先，缓冲区头是一个很大且不易控制的数据结构体（现在是缩减过的了），而且缓冲区头对数据的操作既不方便也不清晰。对内核来说，它更倾向于操作页面结构，因为页面操作起来更为简便，同时效率也高。使用一个巨大的缓冲区头表示每一个独立的缓冲区（可能比页面小）效率低下，所以在 2.6 版本中，许多 I/O 操作都是通过内核直接对页面或地址空间进行操作来完成，不再使用缓冲区头了。这其中所做的一些工作会在第 16 章中进行讨论，具体情况请参考 `address_space` 结构和 `pdflush` 等守护进程（daemon）部分。

缓冲区头带来的第二个弊端是：它仅能描述单个缓冲区，当作为所有 I/O 的容器使用时，缓冲区头会促使内核把对大块数据的 I/O 操作（比如写操作）分解为对多个 `buffer_head` 结构体进行操作。这样做必然会造成不必要的负担和空间浪费。所以 2.5 开发版内核的主要目标就是为块 I/O 操作引入一种新型、灵活并且轻量级的容器，也就是 14.3 节要介绍的 `bio` 结构体。

14.3 bio 结构体

目前内核中块 I/O 操作的基本容器由 `bio` 结构体表示，它定义在文件 `<linux/bio.h>` 中。该结构体代表了正在现场的（活动的）以片断（segment）链表形式组织的块 I/O 操作。一个片段是一小块连续的内存缓冲区。这样的话，就不需要保证单个缓冲区一定要连续。所以通过用片段来描述缓冲区，即使一个缓冲区分散在内存的多个位置上，`bio` 结构体也能对内核保证 I/O 操作的执行。像这样的向量 I/O 就是所谓的聚散 I/O。

`bio` 结构体定义于 `<linux/bio.h>` 中，下面给出 `bio` 结构体和各个域的描述。


```

struct bio {
    sector_t                bi_sector;           /* 磁盘上相关的扇区 */
    struct bio              *bi_next;           /* 请求链表 */
    struct block_device      *bi_bdev;          /* 相关的块设备 */
    unsigned long            bi_flags;          /* 状态和命令标志 */
    unsigned long            bi_rw;             /* 读还是写 */
    unsigned short           bi_vcnt;           /* bio_vecs 偏移的个数 */
    unsigned short           bi_idx;            /* bio_io_vect 的当前索引 */
    unsigned short           bi_phys_segments;  /* 结合后的片断数目 */
    unsigned int             bi_size;           /* I/O 计数 */
    unsigned int             bi_seg_front_size; /* 第一个可合并的段大小 */
    unsigned int             bi_seg_back_size;  /* 最后一个可合并的段大小 */
    unsigned int             bi_max_vecs;       /* bio_vecs 数目上限 */
    unsigned int             bi_comp_cpu;       /* 结束 CPU */
    atomic_t                 bi_cnt;            /* 使用计数 */
    struct bio_vec           *bi_io_vec;        /* bio_vecs 链表 */
    bio_end_io_t             *bi_end_io;        /* I/O 完成方法 */
    void                    *bi_private;        /* 拥有者的私有方法 */
    bio_destructor_t         *bi_destructor;    /* 撤销方法 */
    struct bio_vec           bi_inline_vecs[0]; /* 内嵌 bio 向量 */
};

```

使用 bio 结构体的目的主要是代表正在现场执行的 I/O 操作，所以该结构体中的主要域都是用来管理相关信息的，其中最重要的几个域是 bi_io_vecs、bi_vcnt 和 bi_idx。图 14-2 显示了 bio 结构体及其他结构体之间的关系。

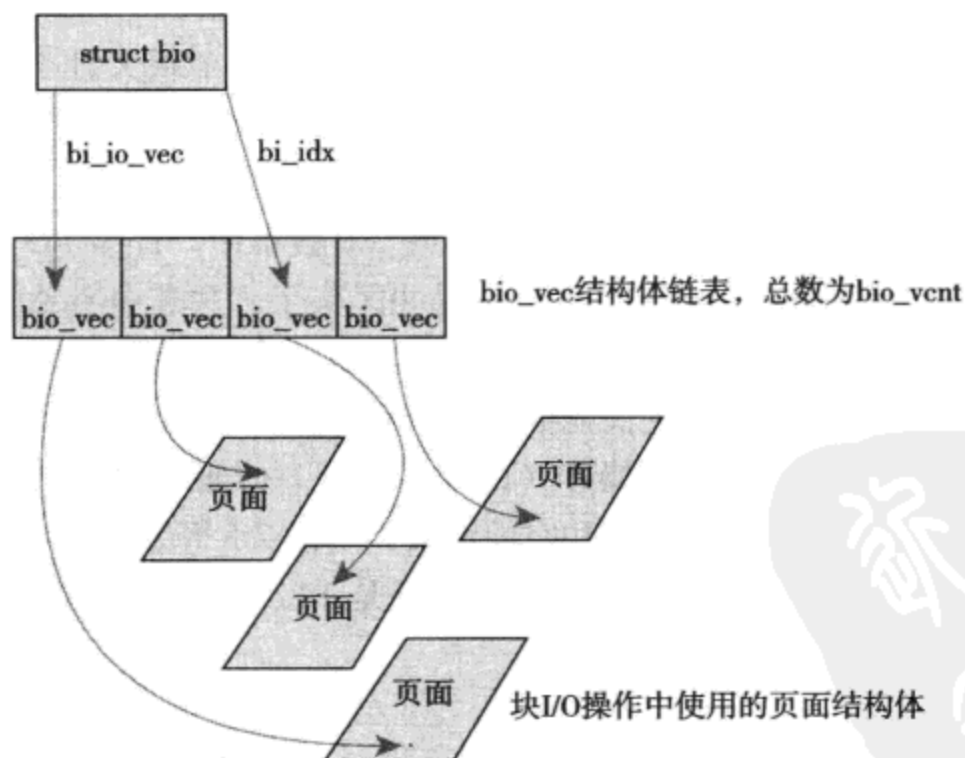


图 14-2 bio 结构体、bio_vec 结构体和 page 结构体之间的关系

14.3.1 I/O 向量

bi_io_vec 域指向一个 bio_vec 结构体数组，该结构体链表包含了一个特定 I/O 操作所需要使用的片断。每个 bio_vec 结构都是一个形式为 `<page, offset, len>` 的向量，它描述的是一个特定的片断：片断所在的物理页、块在物理页中的偏移位置、从给定偏移量开始的块长度。整个

bio_io_vec 结构体数组表示了一个完整的缓冲区。bio_vec 结构定义在 <linux/bio.h> 文件中：

```
struct bio_vec {
    /* 指向这个缓冲区所驻留的物理页 */
    struct page    *bv_page;

    /* 这个缓冲区以字节为单位的大小 */
    unsigned int    bv_len;

    /* 缓冲区所驻留的页中以字节为单位的偏移量 */
    unsigned int    bv_offset;
};
```

在每个给定的块 I/O 操作中，bi_vcnt 域用来描述 bi_io_vec 所指向的 bio_vec 数组中的向量数目。当块 I/O 操作执行完毕后，bi_idx 域指向数组的当前索引。

总而言之，每一个块 I/O 请求都通过一个 bio 结构体表示。每个请求包含一个或多个块，这些块存储在 bio_vec 结构体数组中。这些结构体描述了每个片段在物理页中的实际位置，并且像向量一样被组织在一起。I/O 操作的第一个片段由 b_io_vec 结构体所指向，其他的片段在其后依次放置，共有 bi_vcnt 个片段。当块 I/O 层开始执行请求、需要使用各个片段时，bi_idx 域会不断更新，从而总指向当前片段。

bi_idx 域指向数组中的当前 bio_vec 片段，块 I/O 层通过它可以跟踪块 I/O 操作的完成进度。但该域更重要的作用在于分割 bio 结构体。像冗余廉价磁盘阵列 (RAID，出于提高性能和可靠性的目的，将单个磁盘的卷扩展到多个磁盘上) 这样的驱动器可以把单独的 bio 结构体 (原本是为单个设备使用准备的)，分割到 RAID 阵列中的各个硬盘上去。RAID 设备驱动只需要拷贝这个 bio 结构体，再把 bi_idx 域设置为每个独立硬盘操作时需要的位置就可以了。

bi_cnt 域记录 bio 结构体的使用计数，如果该域值减为 0，就应该撤销该 bio 结构体，并释放它占用的内存。通过下面两个函数管理使用计数。

```
void bio_get(struct bio *bio)
void bio_put(struct bio *bio)
```

前者增加使用计数，后者减少使用计数 (如果计数减到 0，则撤销 bio 结构体)。在操作正在活动的 bio 结构体时，一定要首先增加它的使用计数，以免在操作过程中该 bio 结构体被释放；相反，在操作完毕后，要减少使用计数。

最后要说明的是 bi_private 域，这是一个属于拥有者 (也就是创建者) 的私有域，只有创建了 bio 结构的拥有者可以读写该域。

14.3.2 新老方法对比

缓冲区头和新的 bio 结构体之间存在显著差别。bio 结构体代表的是 I/O 操作，它可以包括内存中的一个或多个页；而另一方面，buffer_head 结构体代表的是一个缓冲区，它描述的仅仅是磁盘中的一个块。因为缓冲区头关联的是单独页中的单独磁盘块，所以它可能会引起不必要的分割，将请求按块为单位划分，只能靠以后才能再重新组合。由于 bio 结构体是轻量级的，它描述的块可以不需要连续存储区，并且不需要分割 I/O 操作。

利用 bio 结构体代替 buffer_head 结构体还有以下好处：

- bio 结构体很容易处理高端内存，因为它处理的是物理页而不是直接指针。
- bio 结构体既可以代表普通页 I/O，同时也可以代表直接 I/O（指那些不通过页高速缓存的 I/O 操作——请参考第 16 章中对页高速缓存的讨论）。
- bio 结构体便于执行分散-集中（矢量化）的块 I/O 操作，操作中的数据可取自多个物理页面。
- bio 结构体相比缓冲区头属于轻量级的结构体。因为它只需要包含块 I/O 操作所需的信息就行了，不用包含与缓冲区本身相关的不必要信息。

但是还是需要缓冲区头这个概念，毕竟它还负责描述磁盘块到页面的映射。bio 结构体不包含任何和缓冲区相关的状态信息——它仅仅是一个矢量数组，描述一个或多个单独块 I/O 操作的数据片段和相关信息。在当前设置中，当 bio 结构体描述当前正在使用的 I/O 操作时，buffer_head 结构体仍然需要包含缓冲区信息。内核通过这两种结构分别保存各自的信息，可以保证每种结构所含的信息量尽可能地少。

14.4 请求队列

块设备将它们挂起的块 I/O 请求保存在请求队列中，该队列由 request_queue 结构体表示，定义在文件 <linux/blkdev.h> 中，包含一个双向请求链表以及相关控制信息。通过内核中像文件系统这样高层的代码将请求加入到队列中。请求队列只要不为空，队列对应的块设备驱动程序就会从队列头获取请求，然后将其送入对应的块设备上去。请求队列表中的每一项都是一个单独的请求，由 request 结构体表示。

队列中的请求由结构体 request 表示，它定义在文件 <linux/blkdev.h> 中。因为一个请求可能要操作多个连续的磁盘块，所以每个请求可以由多个 bio 结构体组成。注意，虽然磁盘上的块必须连续，但是在内存中这些块并不一定要连续——每个 bio 结构体都可以描述多个片段（回忆一下，片段是内存中连续的小区域），而每个请求也可以包含多个 bio 结构体。

14.5 I/O 调度程序

如果简单地以内核产生请求的次序直接将请求发向块设备的话，性能肯定让人难以接受。磁盘寻址是整个计算机中最慢的操作之一，每一次寻址（定位硬盘磁头到特定块上的某个位置）需要花费不少时间。所以尽量缩短寻址时间无疑是提高系统性能的关键。

为了优化寻址操作，内核既不会简单地按请求接收次序，也不会立即将其提交给磁盘。相反，它会在提交前，先执行名为合并与排序的预操作，这种预操作可以极大地提高系统的整体性能^①。在内核中负责提交 I/O 请求的子系统称为 I/O 调度程序。

I/O 调度程序将磁盘 I/O 资源分配给系统中所有挂起的块 I/O 请求。具体地说，这种资源分配是通过将请求队列中挂起的请求合并和排序来完成的。注意不要将 I/O 调度程序和进程调度程序（请看第 4 章）混淆。进程调度程序的作用是将处理器资源分配给系统中的运行进程。这两种

① 这一点需要强调。如果一个系统没有这些功能，或者这些功能实现得很差，那么即使是数量不大的块 I/O 操作，执行性能也会很糟糕。

子系统看起来非常相似，但并不相同。进程调度程序和 I/O 调度程序都是将一个资源虚拟给多个对象，对进程调度程序来说，处理器被虚拟并被系统中的运行进程共享。这种虚拟提供给用户的就是多任务和分时操作系统，像 Unix 系统。相反，I/O 调度程序虚拟块设备给多个磁盘请求，以便降低磁盘寻址时间，确保磁盘性能的最优化。

14.5.1 I/O 调度程序的工作

I/O 调度程序的工作是管理块设备的请求队列。它决定队列中的请求排列顺序以及在什么时候派发请求到块设备。这样做有利于减少磁盘寻址时间，从而提高全局吞吐量。注意，全局这个定语很重要，坦率地讲，一个 I/O 调度器可能为了提高系统整体性能，而对某些请求不公。

I/O 调度程序通过两种方法减少磁盘寻址时间：合并与排序。合并指将两个或多个请求结合成一个新请求。考虑一下这种情况，文件系统提交请求到请求队列——从文件中读取一个数据区（当然，最终所有的操作都是针对扇区和块进行的，而不是文件，还假定请求的块都是来自文件块），如果这时队列中已经存在一个请求，它访问的磁盘扇区和当前请求访问的磁盘扇区相邻（比如，同一个文件中早些时候被读取的数据区），那么这两个请求就可以合并为一个对单个和多个相邻磁盘扇区操作的新请求。通过合并请求，I/O 调度程序将多次请求的开销压缩成一次请求的开销。更重要的是，请求合并后只需要传递给磁盘一条寻址命令，就可以访问到请求合并前必须多次寻址才能访问完的磁盘区域了，因此合并请求显然能减少系统开销和磁盘寻址次数。

现在，假设在读请求被提交给请求队列的时候，队列中并不需要操作相邻扇区的其他请求，此时就无法将当前请求与其他请求合并，当然，可以将其插入请求队列的尾部。但是如果有其他请求需要操作磁盘上类似的位置呢？如果存在一个请求，它要操作的磁盘扇区位置与当前请求比较接近，那么是不是该让这两个请求在请求队列上也相邻呢？事实上，I/O 调度程序的确是这样处理上述情况的，整个请求队列将按扇区增长方向有序排列。使所有请求按硬盘上扇区的排列顺序有序排列（尽可能的）的目的不仅是为了缩短单独一次请求的寻址时间，更重要的优化在于，通过保持磁盘头以直线方向移动，缩短了所有请求的磁盘寻址时间。该排序算法类似于电梯调度——电梯不能随意地从一层跳到另一层，它应该向一个方向移动，当抵达了同一方向上的最后一层后，再掉头向另一个方向移动。出于这种相似性，所以 I/O 调度程序（或这种排序算法）称作电梯调度。

14.5.2 Linux 电梯

下面看看 Linux 中实际使用的 I/O 调度程序。我们看到的第一个 I/O 调度程序称为 Linux 电梯（没错，Linux 确实是用他的名字命名了这个电梯）。在 2.4 版内核中，Linux 电梯是默认的 I/O 调度程序。虽然后来在 2.6 版内核中它被另外两种调度程序取代了，但是由于这个电梯比后来的调度程序简单，而且它们执行的许多功能都相似，所以它可以作为一个优秀的入门介绍程序。

Linux 电梯能执行合并与排序预处理。当有新的请求加入队列时，它首先会检查其他每一个挂起的请求是否可以和新请求合并。Linux 电梯 I/O 调度程序可以执行向前和向后合并，合并类型描述的是请求向前面还是向后面，这一点和已有请求相连。如果新请求正好连在一个现存的请求前，就是向前合并；相反如果新请求直接连在一个现存的请求后，就是向后合并。鉴于文件的分布（通常以扇区号的增长表现）特点和 I/O 操作执行方式具有典型性（一般都是从头读向尾，

很少从反方向读), 所以向前合并相比向后合并要少得多, 但是 Linux 电梯还是会对两种合并类型都进行检查。

如果合并尝试失败, 那么就需要寻找可能的插入点 (新请求在队列中的位置必须符合请求以扇区方向有序排序的原则)。如果找到, 新请求将被插入到该点; 如果没有合适的位置, 那么新请求就被加入到队列尾部。另外, 如果发现队列中有驻留时间过长的请求, 那么新请求也将被加入到队列尾部, 即使插入后还要排序。这样做是为了避免由于访问相近磁盘位置的请求太多, 而造成访问磁盘其他位置的请求难以得到执行机会这一问题。不幸的是, 这种“年龄”检测方法并不很有效, 因为它并非是给等待了一段时间的请求提供实质性服务, 它仅仅是在经过了一段时间后停止插入—排序请求, 这改善了等待时间但最终还是会导致请求饥饿现象的发生, 所以这是一个 2.4 内核 I/O 调度程序中必须要修改的缺陷。

总而言之, 当一个请求加入到队列中时, 有可能发生四种操作, 它们依次是:

- 1) 如果队列中已存在一个对相邻磁盘扇区操作的请求, 那么新请求将和这个已经存在的请求合并成一个请求。
- 2) 如果队列中存在一个驻留时间过长的请求, 那么新请求将被插入到队列尾部, 以防止其他旧的请求饥饿发生。
- 3) 如果队列中以扇区方向为序存在合适的插入位置, 那么新的请求将被插入到该位置, 保证队列中的请求是以被访问磁盘物理位置为序进行排列的。
- 4) 如果队列中不存在合适的请求插入位置, 请求将被插入到队列尾部。

14.5.3 最终期限 I/O 调度程序

最终期限 (deadline) I/O 调度程序是为了解决 Linux 电梯所带来的饥饿问题而提出的。出于减少磁盘寻址时间的考虑, 对某个磁盘区域上的繁重操作, 无疑会使得磁盘其他位置上的操作请求得不到运行机会。实际上, 一个对磁盘同一位置操作的请求流可以造成较远位置的其他请求永远得不到运行机会, 这是一种很不公平的饥饿现象。

更糟糕的是, 普通的请求饥饿还会带来名为写—饥饿—读 (writes-starving-reads) 这种特殊问题。写操作通常是在内核有空时才将请求提交给磁盘的, 写操作完全和提交它的应用程序异步执行; 读操作则恰恰相反, 通常当应用程序提交一个读请求时, 应用程序会发生堵塞直到读请求被满足, 也就是说, 读操作是和提交它的应用程序同步执行的。所以虽然写反应时间 (提交写请求花费的时间) 不会给系统响应速度造成很大影响, 但是读响应时间 (提交读请求花费的时间) 对系统响应时间来说却非同小可。虽然写请求时间对应用程序性能^①带来的影响不大, 但是应用程序却必须等待读请求完成后才能运行其他程序, 所以读操作响应时间对系统的性能非常重要。

问题还可能更严重, 这是因为读请求往往会相互依靠。比如, 要读大量的文件, 每次都是针对一块很小的缓冲区数据区进行读操作, 而应用程序只有将上一个数据区从磁盘中读取并返回之后, 才能继续读取下一个数据区 (或下一个文件)。糟糕的是, 不管是读还是写, 二者都需要读

① 不过, 我们还是打算把写请求无限期地延迟下去, 因为内核想确保数据最终能写到磁盘, 以避免在内存缓冲区中的数据变得越来越多或者太陈旧。

取像索引节点这样的元数据。从磁盘进一步读取这些块会使 I/O 操作串行化。所以如果每一次请求都发生饥饿现象，那么对读取文件的应用程序来说，全部延迟加起来会造成过长的等待时间，让用户无法忍受。综上所述，读操作具有同步性，并且彼此之间往往相互依靠，所以读请求响应时间直接影响系统性能，因此 2.6 版本内核新引入了最后期限 I/O 调度程序来减少请求饥饿现象，特别是读请求饥饿现象。

注意，减少请求饥饿必须以降低全局吞吐量为代价。Linux 电梯调度程序虽然也做了这样的折中，但显然不够——Linux 电梯可以提供更好的系统吞吐量（通过最小化寻址），可是它总按照扇区顺序将请求插入到队列，从不检查驻留时间过长的请求，更不会将请求插入到列队尾部，所以它虽然能让寻址时间最短，但是却会带来同样不可取的请求饥饿问题。为了避免饥饿同时提供良好的全局吞吐量，最后期限 I/O 调度程序做了更多的努力。既要尽量提高全局吞吐量，又要使请求得到公平处理，这是很困难的。

在最后期限 I/O 调度程序中，每个请求都有一个超时时间。默认情况下，读请求的超时时间是 500ms，写请求的超时时间是 5s。最后期限 I/O 调度请求类似于 Linux 电梯，也以磁盘物理位置为次序维护请求队列，这个队列称为排序队列。当一个新请求递交给排序队列时，最后期限 I/O 调度程序在执行合并和插入请求时类似于 Linux 电梯^①，但是最后期限 I/O 调度程序同时也会以请求类型为依据将它们插入到额外队列中。读请求按次序被插入到特定的读 FIFO 队列中，写请求被插入到特定的写 FIFO 队列中。虽然普通队列以磁盘扇区为序进行排列，但是这些队列是以 FIFO（很有效，以时间为基准排序）形式组织的，结果新队列总是被加入到队列尾部。对于普通操作来说，最后期限 I/O 调度程序将请求从排序队列的头部取下，再推入到派发队列中，派发队列然后将请求提交给磁盘驱动，从而保证了最小化的请求寻址。

如果在写 FIFO 队列头，或是在读 FIFO 队列头的请求超时（也就是，当前时间超过了请求指定的超时时间），那么最后期限 I/O 调度程序便从 FIFO 队列中提取请求进行服务。依靠这种方法，最后期限 I/O 调度程序试图保证不会发生有请求在明显超期的情况下仍不能得到服务的现象，参见图 14-3。



图 14-3 最后期限 I/O 调度程序的三个队列

注意，最后期限 I/O 调度算法并不能严格保证请求的响应时间，但是通常情况下，可以在请求超时或超时前提交和执行，以防止请求饥饿现象的发生。由于读请求给定的超时时间要比写请求短许多，所以最后期限 I/O 调度器也确保了写请求不会因为堵塞读请求而使读请求发生饥饿。这种对读操作的照顾确保了读响应时间尽可能短。

^① 最后期限 I/O 排序执行向前合并是一个可选项。因为读操作请求通常很少需要向前合并，所以向前合并通常不必考虑。

最后期限 I/O 调度程序的实现在文件 `block/deadline-iosched.c` 中。

14.5.4 预测 I/O 调度程序

虽然最后期限 I/O 调度程序为降低读操作响应时间做了许多工作，但是它同时也降低了系统吞吐量。假设一个系统处于很繁重的写操作期间，每次提交读请求，I/O 调度程序都会迅速处理读请求，所以磁盘首先为读操作进行寻址，执行读操作，然后返回再寻址进行写操作，并且对每个读请求都重复这个过程。这种做法对读请求来说是件好事，但是两次寻址操作（一次对读操作定位，一次返回来进行写操作定位）却损害了系统全局吞吐量。预测（Anticipatory）I/O 调度程序的目标就是在保持良好的读响应的同时也能提供良好的全局吞吐量。

预测 I/O 调度的基础仍然是最后期限 I/O 调度程序，所以它们有很多相同之处。预测 I/O 调度程序也实现了三个队列（加上一个派发队列），并为每个请求设置了超时时间，这点与最后期限 I/O 调度程序一样。预测 I/O 调度程序最主要的改进是它增加了预测启发（anticipation-heuristic）能力。

预测 I/O 调度试图减少在进行 I/O 操作期间，处理新到的读请求所带来的寻址数量。和最后期限 I/O 调度程序一样，读请求通常会在超时前得到处理，但是预测 I/O 调度程序的不同之处在于，请求提交后并不直接返回处理其他请求，而是会有意空闲片刻（实际空闲时间可以设置，默认为 6ms）。这几 ms，对应用程序来说是个提交其他读请求的好机会——任何对相邻磁盘位置操作的请求都会立刻得到处理。在等待时间结束后，预测 I/O 调度程序重新返回原来的位置，继续执行以前剩下的请求。

要注意，如果等待可以减少读请求所带来的向后再向前（back-and-forth）寻址操作，那么完全值得花一些时间来等待更多的请求；如果一个相邻的 I/O 请求在等待期到来，那么 I/O 调度程序可以节省两次寻址操作。如果存在愈来愈多的访问同样区域的读请求到来，那么片刻等待无疑会避免大量的寻址操作。

当然，如果没有 I/O 请求在等待期到来，那么预测 I/O 调度程序会给系统性能带来轻微的损失，浪费掉几 ms。预测 I/O 调度程序所能带来的优势取决于能否正确预测应用程序和文件系统的行为。这种预测依靠一系列的启发和统计工作。预测 I/O 调度程序跟踪并且统计每个应用程序块 I/O 操作的习惯行为，以便正确预测应用程序的未来行为。如果预测准确率足够高，那么预测调度程序便可以大大减少服务读请求所需的寻址开销，而且同时仍能满足请求所需要的系统响应时间要求。这样的话，预测 I/O 调度程序既减少了读响应时间，又能减少寻址次数和时间，所以说它既缩短了系统响应时间，又提高了系统吞吐量。

预测 I/O 调度程序的实现在文件内核源代码树的 `block/as-iosched.c` 中，它是 Linux 内核中缺省的 I/O 调度程序，对大多数工作负荷来说都执行良好，对服务器也是理想的。不过，在某些非常见而又有严格工作负荷的服务器（包括数据库挖掘服务器）上，这个调度程序执行的效果不好。

14.5.5 完全公正的排队 I/O 调度程序

完全公正的排队 I/O 调度程序（Complete Fair Queuing，CFQ）是为专有工作负荷设计的，不过，在实际中，也为多种工作负荷提供了良好的性能。但是，它与前面介绍的 I/O 调度程序有

根本的不同。

CFQ I/O 调度程序把进入的 I/O 请求放入特定的队列中，这种队列是根据引起 I/O 请求的进程组织的。例如，来自 foo 进程的 I/O 请求进入 foo 队列，而来自 bar 进程的 I/O 请求进入 bar 队列。在每个队列中，刚进入的请求与相邻请求合并在一起，并进行插入分类。队列由此按扇区方式分类，这与其他 I/O 调度程序队列类似。CFQ I/O 调度程序的差异在于每一个提交 I/O 的进程都有自己的队列。

CFQ I/O 调度程序以时间片轮转调度队列，从每个队列中选取请求数（默认值为 4，可以进行配置），然后进行下一轮调度。这就在进程级提供了公平，确保每个进程接收公平的磁盘带宽片断。预定的工作负荷是多媒体，在这种媒体中，这种公平的算法可以得到保证，比如，音频播放器总能够及时从磁盘再填满它的音频缓冲区。不过，实际上，CFQ I/O 调度程序在很多场合都能很好地执行。

完全公正的排队 I/O 调度程序位于 block/cfq-iosched.c。尽管这主要推荐给桌面工作负荷使用，但是，如果没有其他异常情况，它在几乎所有的工作负荷中都能很好地执行。

14.5.6 空操作的 I/O 调度程序

第四种也是最后一种 I/O 调度程序是空操作（Noop）I/O 调度程序，之所以这样命名是因为它基本上是一个空操作，不做多少事情。空操作 I/O 调度程序不进行排序，或者也不进行什么其他形式的预寻址操作。依此类推，它也没必要实现那些老套的算法，也就是在以前的 I/O 调度程序中看到的为了最小化请求周期而采用的算法。

不过，空操作 I/O 调度程序忘不了执行合并，这就像它的家务事。当一个新的请求提交到队列时，就把它与任一相邻的请求合并。除了这一操作，空操作 I/O 调度程序的确再不做什么，只是维护请求队列以近乎 FIFO 的顺序排列，块设备驱动程序便可以从这种队列中摘取请求。

空操作 I/O 调度程序不勤奋工作是有道理的。因为它打算用在块设备，那是真正的随机访问设备，比如闪存卡。如果块设备只有一点或者没有“寻道”的负担，那么，就没有必要对进入的请求进行插入排序，因此，空操作 I/O 调度程序是理想的候选者。

空操作 I/O 调度程序位于 block/noop-iosched.c，它是专为随机访问设备而设计的。

14.5.7 I/O 调度程序的选择

你已经看到 2.6 内核中四种不同的 I/O 调度程序。其中的每一种 I/O 调度程序都可以被启用，并内置在内核中。作为缺省，块设备使用完全公平的 I/O 调度程序。在启动时，可以通过命令行选项 `elevator=foo` 来覆盖缺省，这里 foo 是一个有效而激活的 I/O 调度程序，参看表 14-2。

表 14-2 给定 elevator 选项的参数

参 数	I/O 调度程序
as	预测
cfq	完全公正的排队
deadline	最终期限
noop	空操作

例如，内核命令行选项 `elevator=as` 会启用预测 I/O 调度程序给所有的块设备，从而覆盖默认的完全公正调度程序。

14.6 小结

在本章中，我们讨论了块设备的基本知识，并考察了块 I/O 层所用的数据结构：`bio`，表示活动的 I/O 操作；`buffer_head`，表示块到页的映射；还有请求结构，表示具体的 I/O 请求。我们追寻了 I/O 请求简单但重要的生命历程，其生命的重要点就是 I/O 调度程序。我们讨论了 I/O 调度程序所涉及的困惑问题，同时仔细推敲了当前内核的 4 种 I/O 调度程序，以及 2.4 版本中原有的 `linux` 电梯调度。

我们将在第 15 章讨论进程地址空间。



第 15 章

进程地址空间

第 12 章介绍了内核如何管理物理内存。其实内核除了管理本身的内存外，还必须管理用户空间中进程的内存。我们称这个内存为进程地址空间，也就是系统中每个用户空间进程所看到的内存。Linux 操作系统采用虚拟内存技术，因此，系统中的所有进程之间以虚拟方式共享内存。对一个进程而言，它好像都可以访问整个系统的所有物理内存。更重要的是，即使单独一个进程，它拥有的地址空间也可以远远大于系统物理内存。本章将集中讨论内核如何管理进程地址空间。

15.1 地址空间

进程地址空间由进程可寻址的虚拟内存组成，而且更为重要的特点是内核允许进程使用这种虚拟内存中的地址。每个进程都有一个 32 位或 64 位的平坦（flat）地址空间，空间的具体大小取决于体系结构。术语“平坦”指的是地址空间范围是一个独立的连续区间（比如，地址从 0 扩展到 4294967295 的 32 位地址空间）。一些操作系统提供了段地址空间，这种地址空间并非是一个独立的线性区域，而是被分段的，但现代采用虚拟内存的操作系统通常都使用平坦地址空间而不是分段式的内存模式。通常情况下，每个进程都有唯一的这种平坦地址空间。一个进程的地址空间与另一个进程的地址空间即使有相同的内存地址，实际上也彼此互不相干。我们称这样的进程为线程。

内存地址是一个给定的值，它要在地址空间范围之内，比如 4021f000。这个值表示的是进程 32 位地址空间中的一个特定的字节。尽管一个进程可以寻址 4GB 的虚拟内存（在 32 位的地址空间中），但这并不代表它就有权访问所有的虚拟地址。在地址空间中，我们更为关心的是一些虚拟内存的地址区间，比如 08048000-0804c000，它们可被进程访问。这些可被访问的合法地址空间称为内存区域（memory areas）。通过内核，进程可以给自己的地址空间动态地添加或减少内存区域。

进程只能访问有效内存区域内的内存地址。每个内存区域也具有相关权限如对相关进程有可读、可写、可执行属性。如果一个进程访问了不在有效范围中的内存区域，或以不正确的方式访问了有效地址，那么内核就会终止该进程，并返回“段错误”信息。

内存区域可以包含各种内存对象，比如：

- 可执行文件代码的内存映射，称为代码段（text section）。
- 可执行文件的已初始化全局变量的内存映射，称为数据段（data section）。

- 包含未初始化全局变量，也就是 bss 段[⊖]的零页（页面中的信息全部为 0 值，所以可用于映射 bss 段等目的）的内存映射。
- 用于进程用户空间栈（不要和进程内核栈混淆，进程的内核栈独立存在并由内核维护）的零页的内存映射。
- 每一个诸如 C 库或动态连接程序等共享库的代码段、数据段和 bss 也会被载入进程的地址空间。
- 任何内存映射文件。
- 任何共享内存段。
- 任何匿名的内存映射，比如由 malloc()[⊖]分配的内存。

进程地址空间中的任何有效地址都只能位于唯一的区域，这些内存区域不能相互覆盖。可以看到，在执行的进程中，每个不同的内存片段都对应一个独立的内存区域：栈、对象代码、全局变量、被映射的文件等。

15.2 内存描述符

内核使用内存描述符结构体表示进程的地址空间，该结构包含了和进程地址空间有关的全部信息。内存描述符由 mm_struct 结构体表示，定义在文件 <linux/sched.h> 中。下面给出内存描述符的结构和各个域的描述：

```

struct mm_struct{
    struct vm_area_struct    *mmap;           /* 内存区域链表 */
    struct rb_root           mm_rb;           /* VMA 形成的红黑树 */
    struct vm_area_struct    *mmap_cache;     /* 最近使用的内存区域 */
    unsigned long            free_area_cache; /* 地址空间第一个空洞 */
    pgd_t                    *pgd;           /* 页全局目录 */
    atomic_t                 mm_users;        /* 使用地址空间的用户数 */
    atomic_t                 mm_count;        /* 主使用计数器 */
    int                      map_count;       /* 内存区域的个数 */
    struct rw_semaphore      mmap_sem;        /* 内存区域的信号量 */
    spinlock_t               page_table_lock; /* 页表锁 */
    struct list_head         mmlist;          /* 所有 mm_struct 形成的链表 */
    unsigned long            start_code;      /* 代码段的开始地址 */
    unsigned long            end_code;        /* 代码段的结束地址 */
    unsigned long            start_data;      /* 数据的首地址 */
    unsigned long            end_data;        /* 数据的尾地址 */
    unsigned long            start_brk;       /* 堆的首地址 */
    unsigned long            brk;            /* 堆的尾地址 */
    unsigned long            start_stack;     /* 进程栈的首地址 */
    unsigned long            arg_start;       /* 命令行参数的首地址 */
    unsigned long            arg_end;        /* 命令行参数的尾地址 */
    unsigned long            env_start;       /* 环境变量的首地址 */
    unsigned long            env_end;        /* 环境变量的尾地址 */
}

```

⊖ 术语“BSS”已经有些年头了，它是 block started by symbol 的缩写。因为未初始化的变量没有对应的值，所以并不需要存放在可执行对象中。但是因为 C 标准强制规定未初始化的全局变量要被赋予特殊的默认值（基本上是 0 值），所以内核要将变量（未赋值的）从可执行代码载入到内存中，然后将零页映射到该片内存上，于是这些未初始化变量就被赋予了 0 值。这样做避免了在目标文件中显式地进行初始化，减少了空间浪费。

⊖ 在最新版本的 glibc 中，通过 mmap() 和 brk() 来实现 malloc() 函数。


```

unsigned long      rss;                /* 所分配的物理页 */
unsigned long      total_vm;           /* 全部页面数目 */
unsigned long      locked_vm;         /* 上锁的页面数目 */
unsigned long      saved_auxv[AT_VECTOR_SIZE]; /* 保存的 auxv */
cpumask_t          cpu_vm_mask;       /* 懒惰 (lazy) TLB 交换掩码 */
mm_context_t       context;           /* 体系结构特殊数据 */
unsigned long      flags;             /* 状态标志 */
int               core_waiters;       /* 内核转储等待线程 */
struct core_state  *core_state;       /* 核心转储的支持 */
spinlock_t         ioctx_lock;        /* AIO I/O 链表锁 */
struct hlist_head  ioctx_list;        /* AIO I/O 链表 */
};

```

mm_users 域记录正在使用该地址的进程数目。比如，如果两个线程共享该地址空间，那么 mm_users 的值便等于 2；mm_count 域是 mm_struct 结构体的主引用计数。所有的 mm_users 都等于 mm_count 的增加量。这样，在前面的例子中，mm_count 就仅仅为 1。如果有 9 个线程共享某个地址空间，那么 mm_users 将会是 9，而 mm_count 的值将再次为 1。当 mm_users 的值减为 0（即所有正使用该地址空间的线程都退出）时，mm_count 域的值才变为 0。当 mm_count 的值等于 0，说明已经没有任何指向该 mm_struct 结构体的引用了，这时该结构体会被撤销。当内核在一个地址空间上操作，并需要使用与该地址相关联的引用计数时，内核便增加 mm_count。内核同时使用这两个计数器是为了区别主使用计数（mm_count）器和使用该地址空间的进程的数目（mm_users）。

mmap 和 mm_rb 这两个不同数据结构体描述的对象是相同的：该地址空间中的全部内存区域。但是前者以链表形式存放而后者以红—黑树的形式存放。红—黑树是一种二叉树，与其他二叉树一样，搜索它的时间复杂度为 $O(\log n)$ 。在本章后续部分“内存区域的树型结构和内存区域的链表结构”一节中，我们将进一步讨论红—黑树。

内核通常会避免使用两种数据结构组织同一种数据，但此处内核这样的冗余确实派得上用场。mmap 结构体作为链表，利于简单、高效地遍历所有元素；而 mm_rb 结构体作为红—黑树，更适合搜索指定元素。对内存区域的具体操作将在本章的后续部分详细介绍。内核并没有复制 mm_struct 结构体，而仅仅被包含其中。覆盖树上的链表并用这两个结构体同时访问相同的数据集，有时候我们将此操作称作线索树。

所有的 mm_struct 结构体都通过自身的 mm_list 域连接在一个双向链表中，该链表的首元素是 init_mm 内存描述符，它代表 init 进程的地址空间。另外要注意，操作该链表的时候需要使用 mm_list_lock 锁来防止并发访问，该锁定义在文件 kernel/fork.c 中。

15.2.1 分配内存描述符

在进程的进程描述符（在 <linux/sched.h> 中定义的 task_struct 结构体就表示进程描述符）中，mm 域存放着该进程使用的内存描述符，所以 current->mm 便指向当前进程的内存描述符。fork() 函数利用 copy_mm() 函数复制父进程的内存描述符，也就是 current->mm 域给其子进程，而子进程中的 mm_struct 结构体实际是通过文件 kernel/fork.c 中的 allocate_mm() 宏从 mm_cachep slab 缓存中分配得到的。通常，每个进程都有唯一的 mm_struct 结构体，即唯一的

进程地址空间。

如果父进程希望和其子进程共享地址空间，可以在调用 `clone()` 时，设置 `CLONE_VM` 标志。我们把这样的进程称作线程。回忆第 3 章，是否共享地址空间几乎是进程和 Linux 中所谓的线程间本质上的唯一区别。除此以外，Linux 内核并不区别对待它们，线程对内核来说仅仅是一个共享特定资源的进程而已。

当 `CLONE_VM` 被指定后，内核就不再需要调用 `allocate_mm()` 函数了，而仅仅需要在调用 `copy_mm()` 函数中将 `mm` 域指向其父进程的内存描述符就可以了：

```
if(clone_flags & CLONE_VM){
    /*
     * current 是父进程而 tsk 在 fork() 执行期间是子进程
     */
    atomic_inc(&current->mm->mm_users);
    tsk->mm= current ->mm;
}
```

15.2.2 撤销内存描述符

当进程退出时，内核会调用定义在 `kernel/exit.c` 中的 `exit_mm()` 函数，该函数执行一些常规的撤销工作，同时更新一些统计量。其中，该函数会调用 `mmapput()` 函数减少内存描述符中的 `mm_users` 用户计数，如果用户计数降到零，将调用 `mmdrop()` 函数，减少 `mm_count` 使用计数。如果使用计数也等于零了，说明该内存描述符不再有任何使用者了，那么调用 `free_mm()` 宏通过 `kmem_cache_free()` 函数将 `mm_struct` 结构体归还到 `mm_cache` slab 缓存中。

15.2.3 mm_struct 与内核线程

内核线程没有进程地址空间，也没有相关的内存描述符。所以内核线程对应的进程描述符中 `mm` 域为空。事实上，这也正是内核线程的真实含义——它们没有用户上下文。

省了进程地址空间再好不过了，因为内核线程并不需要访问任何用户空间的内存（那它们访问谁的呢？）而且因为内核线程在用户空间中没有任何页，所以实际上它们并不需要有自己的内存描述符和页表（后面的内容将讲述页表）。尽管如此，即使访问内核内存，内核线程也还是需要使用一些数据的，比如页表。为了避免内核线程为内存描述符和页表浪费内存，也为了当新内核线程运行时，避免浪费处理器周期向新地址空间进行切换，内核线程将直接使用前一个进程的内存描述符。

当一个进程被调度时，该进程的 `mm` 域指向的地址空间被装载到内存，进程描述符中的 `active_mm` 域会被更新，指向新的地址空间。内核线程没有地址空间，所以 `mm` 域为 `NULL`。于是，当一个内核线程被调度时，内核发现它的 `mm` 域为 `NULL`，就会保留前一个进程的地址空间，随后内核更新内核线程对应的进程描述符中的 `active_mm` 域，使其指向前一个进程的内存描述符。所以在需要时，内核线程便可以使用前一个进程的页表。因为内核线程不访问用户空间的内存，所以它们仅仅使用地址空间中和内核内存相关的信息，这些信息的含义和普通进程完全相同。

15.3 虚拟内存区域

内存区域由 `vm_area_struct` 结构体描述，定义在文件 `<linux/mm_types.h>` 中。内存区域在 Linux 内核中也经常称作虚拟内存区域（virtual memory Areas, VMAs）。

`vm_area_struct` 结构体描述了指定地址空间内连续区间上的一个独立内存范围。内核将每个内存区域作为一个单独的内存对象管理，每个内存区域都拥有一致的属性，比如访问权限等，另外，相应的操作也都一致。按照这样的方式，每一个 VMA 就可以代表不同类型的内存区域（比如内存映射文件或者进程用户空间栈），这种管理方式类似于使用 VFS 层的面向对象方法（请看第 13 章），下面给出该结构定义和各个域的描述：

```
struct vm_area_struct {
    struct mm_struct      *vm_mm;           /* 相关的 mm_struct 结构体 */
    unsigned long         vm_start;         /* 区间的首地址 */
    unsigned long         vm_end;           /* 区间的尾地址 */
    struct vm_area_struct *vm_next;         /* VMA 链表 */
    pgprot_t              vm_page_prot;     /* 访问控制权限 */
    unsigned long         vm_flags;         /* 标志 */
    struct rb_node         vm_rb;           /* 树上该 VMA 的节点 */
    union {                          /* 或者是关联于 address_space->i_mmap 字段，或者是关联于
        address_space->i_mmap_nonlinear 字段 */
        struct {
            struct list_head list;
            void *parent;
            struct vm_area_struct *head;
        } vm_set;
        struct prio_tree_node prio_tree_node;
    } shared;
    struct list_head      anon_vma_node;    /* anon_vma 项 */
    struct anon_vma       *anon_vma;       /* 匿名 VMA 对象 */
    struct vm_operations_struct *vm_ops;    /* 相关的操作表 */
    unsigned long         vm_pgoff;         /* 文件中的偏移量 */
    struct file           *vm_file;         /* 被映射的文件（如果存在） */
    void                  *vm_private_data; /* 私有数据 */
};
```

每个内存描述符都对应于进程地址空间中的唯一区间。`vm_start` 域指向区间的首地址（最低地址），`vm_end` 域指向区间的尾地址（最高地址）之后的第一个字节，也就是说，`vm_start` 是内存区间的开始地址（它本身在区间内），而 `vm_end` 是内存区间的结束地址（它本身在区间外），因此，`vm_end - vm_start` 的大小便是内存区间的长度，内存区域的位置就在 `[vm_start, vm_end]` 之中。注意，在同一个地址空间内的不同内存区间不能重叠。

`vm_mm` 域指向和 VMA 相关的 `mm_struct` 结构体，注意，每个 VMA 对其相关的 `mm_struct` 结构体来说都是唯一的，所以即使两个独立的进程将同一个文件映射到各自的地址空间，它们分别都会有一个 `vm_area_struct` 结构体来标志自己的内存区域；反过来，如果两个线程共享一个地址空间，那么它们也同时共享其中的所有 `vm_area_struct` 结构体。

15.3.1 VMA 标志

VMA 标志是一种位标志，其定义见 `<linux/mm.h>`。它包含在 `vm_flags` 域内，标志了内存区

域所包含的页面的行为和信息。和物理页的访问权限不同，VMA 标志反映了内核处理页面所需要遵守的行为准则，而不是硬件要求。而且，`vm_flags` 同时也包含了内存区域中每个页面的信息，或内存区域的整体信息，而不是具体的独立页面。表 15-1 列出了所有 VMA 标志的可能取值。

表 15-1 VMA 标志

标 志	对 VMA 及其页面的影响
VM_READ	页面可读取
VM_WRITE	页面可写
VM_EXEC	页面可执行
VM_SHARED	页面可共享
VM_MAYREAD	VM_READ 标志可被设置
VM_MAYWRITE	VM_WRITE 标志可被设置
VM_MAYEXEC	VM_EXEC 标志可被设置
VM_MAYSHARE	VM_SHARE 标志可被设置
VM_GROWSDOWN	区域可向下增长
VM_GROWSUP	区域可向上增长
VM_SHM	区域可用作共享内存
VM_DENYWRITE	区域映射一个不可写文件
VM_EXECUTABLE	区域映射一个可执行文件
VM_LOCKED	区域中的页面被锁定
VM_IO	区域映射设备 I/O 空间
VM_SEQ_READ	页面可能会被连续访问
VM_RAND_READ	页面可能会被随机访问
VM_DONTCOPY	区域不能在 <code>fork()</code> 时被拷贝
VM_DONTEXPAND	区域不能通过 <code>mremap()</code> 增加
VM_RESERVED	区域不能被换出
VM_ACCOUNT	该区域是一个记账 VM 对象
VM_HUGETLB	区域使用了 <code>hugetlb</code> 页面
VM_NONLINEAR	该区域是非线性映射的

让我们进一步看看其中有趣和重要的几种标志，VM_READ、VM_WRITE 和 VM_EXEC 标志了内存区域中页面的读、写和执行权限。这些标志根据要求组合构成 VMA 的访问控制权限，当访问 VMA 时，需要查看其访问权限。比如进程的对象代码映射区域可能会标志为 VM_READ 和 VM_EXEC，而没有标志为 VM_WRITE；另一方面，可执行对象数据段的映射区域标志为 VM_READ 和 VM_WRITE，而 VM_EXEC 标志对它就毫无意义。也就是说，只读文件数据段的映射区域仅可被标志为 VM_READ。

VM_SHARED 指明了内存区域包含的映射是否可以在多进程间共享，如果该标志被设置，则我们称其为共享映射；如果未被设置，而仅仅只有一个进程可以使用该映射的内容，我们称它为私有映射。

VM_IO 标志内存区域中包含对设备 I/O 空间的映射。该标志通常在设备驱动程序执行

mmap() 函数进行 I/O 空间映射时才被设置，同时该标志也表示该内存区域不能被包含在任何进程的存放转存 (core dump) 中。VM_RESERVED 标志规定了内存区域不能被换出，它也是在设备驱动程序进行映射时被设置。

VM_SEQ_READ 标志暗示内核应用程序对映射内容执行有序的（线性和连续的）读操作；这样，内核可以有选择地执行预读文件。VM_RANDOM_READ 标志的意义正好相反，暗示应用程序对映射内容执行随机的（非有序的）读操作。因此内核可以有选择地减少或彻底取消文件预读，所以这两个标志可以通过系统调用 madvise() 设置，设置参数分别是 MADV_SEQUENTIAL 和 MADV_RANDOM。文件预读是指在读数据时有意地按顺序多读取一些本次请求以外的数据——希望多读的数据能够很快就被用到。这种预读行为对那些顺序读取数据的应用程序有很大的好处，但是如果数据的访问是随机的，那么预读显然就多余了。

15.3.2 VMA 操作

vm_area_struct 结构体中的 vm_ops 域指向与指定内存区域相关的操作函数表，内核使用表中的方法操作 VMA。vm_area_struct 作为通用对象代表了任何类型的内存区域，而操作表描述针对特定的对象实例的特定方法。

操作函数表由 vm_operations_struct 结构体表示，定义在文件 <linux/mm.h> 中：

```
struct vm_operations_struct {
    void (*open)(struct vm_area_struct *);
    void (*close)(struct vm_area_struct *);
    int (*fault)(struct vm_area_struct *, struct vm_fault *);
    int (*page_mkwrite)(struct vm_area_struct *vma, struct vm_fault *vmf);
    int (*access)(struct vm_area_struct *, unsigned long,
                  void *, int, int);
};
```

下面介绍具体方法：

*void open(struct vm_area_struct *area)

当指定的内存区域被加入到一个地址空间时，该函数被调用。

*void close(struct vm_area_struct *area)

当指定的内存区域从地址空间删除时，该函数被调用。

*int fault(struct vm_area_struct *area, struct vm_fault *vmf)

当没有出现在物理内存中的页面被访问时，该函数被页面故障处理调用。

*int page_mkwrite(struct vm_area_struct *area, struct vm_fault *vmf)

当某个页面为只读页面时，该函数被页面故障处理调用。

*int access(struct vm_area_struct *vma, unsigned long address, void
*buf, int len, int write)

当 get_user_pages() 函数调用失败时，该函数被 access_process_vm() 函数调用。

15.3.3 内存区域的树型结构和内存区域的链表结构

上文讨论过，可以通过内存描述符中的 `mmap` 和 `mm_rb` 域之一访问内存区域。这两个域各自独立地指向与内存描述符相关的全体内存区域对象。其实，它们包含完全相同的 `vm_area_struct` 结构体的指针，仅仅组织方法不同。

`mmap` 域使用单独链表连接所有的内存区域对象。每一个 `vm_area_struct` 结构体通过自身的 `vm_next` 域被连入链表，所有的区域按地址增长的方向排序，`mmap` 域指向链表中第一个内存区域，链中最后一个结构体指针指向空。

`mm_rb` 域使用红-黑树连接所有的内存区域对象。`mm_rb` 域指向红-黑树的根节点，地址空间中每一个 `vm_area_struct` 结构体通过自身的 `vm_rb` 域连接到树中。

红-黑树是一种二叉树，树中的每一个元素称为一个节点，最初的节点称为树根。红-黑树的多数节点都由两个子节点：一个左子节点和一个右子节点，不过也有节点只有一个子节点的情况。树末端的节点称为叶子节点，它们没有子节点。红-黑树中的所有节点都遵从：左边节点值小于右边节点值；另外每个节点都被配以红色或黑色（要么红要么黑，所以叫做红-黑树）。分配的规则为：红节点的子节点为黑色，并且树中的任何一条从节点到叶子的路径必须包含同样数目的黑色节点。记住根节点总为红色。红-黑树的搜索、插入、删除等操作的复杂度都为 $O(\log(n))$ 。

链表用于需要遍历全部节点的时候，而红-黑树适用于在地址空间中定位特定内存区域的时候。内核为了内存区域上的各种不同操作都能获得高性能，所以同时使用了这两种数据结构。

15.3.4 实际使用中的内存区域

可以使用 `/proc` 文件系统和 `pmap (1)` 工具查看给定进程的内存空间和其中所含的内存区域。我们来看一个非常简单的用户空间程序的例子，它其实什么也不做，仅仅是为了做说明：

```
int main(int argc, char *argv[])
{
    return 0;
}
```

下面列出该进程地址空间中包含的内存区域。其中有代码段、数据段和 `bss` 段等。假设该进程与 C 库动态连接，那么地址空间中还将分别包含 `libc.so` 和 `ld.so` 对应的上述三种内存区域。此外，地址空间中还要包含进程栈对应的内存区域。

`/proc/<pid>/maps` 的输出显示了该进程地址空间中的全部内存区域：

```
rlove@wolf:~$ cat /proc/1426/maps
00e80000-00faf000 r-xp 00000000 03:01 208530 /lib/tls/libc-2.5.1.so
00faf000-00fb2000 rw-p 0012f000 03:01 208530 /lib/tls/libc-2.5.1.so
00fb2000-00fb4000 rw-p 00000000 00:00 0
08048000-08049000 r-xp 00000000 03:03 439029 /home/rlove/src/example
08049000-0804a000 rw-p 00000000 03:03 439029 /home/rlove/src/example
40000000-40015000 r-xp 00000000 03:01 80276 /lib/ld-2.5.1.so
40015000-40016000 rw-p 00015000 03:01 80276 /lib/ld-2.5.1.so
4001e000-4001f000 rw-p 00000000 00:00 0
bffffe000-c0000000 rwxp fffff000 00:00 0
```

每行数据格式如下：

开始—结束 访问权限 偏移 主设备号：次设备号 i 节点 文件

pmap(1) 工具^①将上述信息以更方便阅读的形式输出：

```
rlove@wolf:~$ pmap 1426
example [1426]
00e80000(1212KB)  r-xp    (03:01 208530)      /lib/tls/libc-2.5.1.so
00faf000(12KB)   rw-p    (03:01 208530)      /lib/tls/libc-2.5.1.so
00fbz000(8KB)    rw-p    (00:000)
08048000(4KB)    r-xp    (03:03 439029)      /home/rlove/src/example
08049000(4KB)    rw-p    (03:03 439029)      /home/rlove/src/example
40000000(84KB)   r-xp    (03:01 80276)       /lib/ld-2.5.1.so
40015000(4KB)    rw-p    (03:01 80276)       /lib/ld-2.5.1.so
4001e000(4KB)    rw-p    (00:00 0)
bffffe000(8KB)   rwxp    (00:00 0)          [stack]
mapped :1340KB   writable/private : 40KB      shared :0KB
```

前三行分别对应 C 库中 libc.so 的代码段、数据段和 bss 段，接下来的两个行为可执行对象的代码段和数据段，再下来三个行为动态连接程序 ld.so 的代码段、数据段和 bss 段，最后一行是进程的栈。

注意，代码段具有我们所要求的可读且可执行权限；另一方面，数据段和 bss（它们都包含全局数据变量）具有可读、可写但不可执行权限。而堆栈则可读、可写，甚至还可执行——虽然这点并不常用到。

该进程的全部地址空间大约为 1340KB，但是只有大约 40KB 的内存区域是可写和私有的。如果一片内存范围是共享的或不可写的，那么内核只需要在内存中为文件（backing file）保留一份映射。对于共享映射来说，这样做没什么特别的，但是对于不可写内存区域也这样做，就有些让人奇怪了。如果考虑到映射区域不可写意味着该区域不可被改变（映射只用来读），就应该清楚只把该映像读入一次是很安全的。所以 C 库在物理内存中仅仅需要占用 1212KB 空间，而不需要为每个使用 C 库的进程在内存中都保存一个 1212KB 的空间。进程访问了 1340KB 的数据和代码空间，然而仅仅消耗了 40KB 的物理内存，可以看出利用这种共享不可写内存的方法节约了大量的内存空间。

注意没有映射文件的内存区域的设备标志为 00:00，索引接点标志也为 0，这个区域就是零页——零页映射的内容全为零。如果将零页映射到可写的内存区域，那么该区域将全被初始化为 0。这是零页的一个重要用处，而 bss 段需要的就是全 0 的内存区域。由于内存未被共享，所以只要一有进程写该处数据，那么该处数据就将被拷贝出来（就是我们所说的写时拷贝），然后才被更新。

每个和进程相关的内存区域都对应于一个 vm_area_struct 结构体。另外进程不同于线程，进程结构体 task_struct 包含唯一的 mm_struct 结构体引用。

15.4 操作内存区域

内核时常需要在某个内存区域上执行一些操作，比如某个指定地址是否包含在某个内存区域

^① pmap(1) 工具将进程的内存区域格式化后显示，虽然结果中的信息和 /proc 中的是一样的信息，但它的输出形式比 /proc 的输出形式更具可读性。在新版本的 procps 包中可以找到这个工具。

中。这类操作非常频繁，另外它们也是 `mmap()` 例程的基础——我们在 15.5 节会讨论 `mmap()` 操作。为了方便执行这类对内存区域的操作，内核定义了许多的辅助函数。

它们都声明在文件 `<linux/mm.h>` 中。

15.4.1 find_vma()

为了找到一个给定的内存地址属于哪一个内存区域，内核提供了 `find_vma()` 函数。该函数定义在文件 `<mm/mmap.c>` 中：

```
struct vm_area_struct * find_vma(struct mm_struct *mm, unsigned long addr);
```

该函数在指定的地址空间中搜索第一个 `vm_end` 大于 `addr` 的内存区域。换句话说，该函数寻找第一个包含 `addr` 或首地址大于 `addr` 的内存区域，如果没有发现这样的区域，该函数返回 `NULL`；否则返回指向匹配的内存区域的 `vm_area_struct` 结构体指针。注意，由于返回的 VMA 首地址可能大于 `addr`，所以指定的地址并不一定就包含在返回的 VMA 中。因为很有可能在对某个 VMA 执行操作后，还有其他更多的操作会对该 VMA 接着进行操作，所以 `find_vma()` 函数返回的结果被缓存在内存描述符的 `mmap_cache` 域中。实践证明，被缓存的 VMA 会有相当好的命中率（实践中大约 30% ~ 40%），而且检查被缓存的 VMA 速度会很快，如果指定的地址不在缓存中，那么必须搜索和内存描述符相关的所有内存区域。这种搜索通过红-黑树进行：

```
struct vm_area_struct * find_vma(struct mm_struct *mm, unsigned long addr)
{
    struct vm_area_struct *vma = NULL;

    if (mm) {
        vma = mm->mmap_cache;
        if (!(vma && vma->vm_end > addr && vma->vm_start <= addr)) {
            struct rb_node *rb_node;

            rb_node = mm->mm_rb.rb_node;
            vma = NULL;
            while (rb_node) {
                struct vm_area_struct * vma_tmp;

                vma_tmp = rb_entry(rb_node,
                                   struct vm_area_struct, vm_rb);
                if (vma_tmp->vm_end > addr) {
                    vma = vma_tmp;
                    if (vma_tmp->vm_start <= addr)
                        break;
                    rb_node = rb_node->rb_left;
                } else
                    rb_node = rb_node->rb_right;
            }
            if (vma)
                mm->mmap_cache = vma;
        }
    }
}
```

```
    }  
  
    return vma;  
}
```

首先，该函数检查 `mmap_cache`，看看缓存的 VMA 是否包含了所需地址。注意简单地检查 VMA 的 `vm_end` 是否大于 `addr`，并不能保证该 VMA 是第一个大于 `addr` 的内存区域，所以缓存要想发挥作用，就要求指定的地址必须包含在被缓存的 VMA 中——幸好，这也正是连续操作同一 VMA 必然发生的情况。

如果缓存中并未包含希望的 VMA，那么该函数必须搜索红-黑树。如果当前 VMA 的 `vm_end` 大于 `addr`，进入左子节点继续搜索；否则，沿右边子节点搜索，直到找到包含 `addr` 的 VMA 为止。如果没有包含 `addr` 的 VMA 被找到，那么该函数继续搜索树，并且返回大于 `addr` 的第一个 VMA。如果也不存在满足要求的 VMA，那该函数返回 `NULL`。

15.4.2 `find_vma_prev()`

`find_vma_prev()` 函数和 `find_vma()` 工作方式相同，但是它返回第一个小于 `addr` 的 VMA。该函数定义和声明分别在文件 `mm/mmap.c` 中和文件 `<linux/mm.h>` 中：

```
struct vm_area_struct * find_vma_prev(struct mm_struct *mm, unsigned long addr ,  
                                     struct vm_area_struct **pprev)
```

`pprev` 参数存放指向先于 `addr` 的 VMA 指针。

15.4.3 `find_vma_intersection()`

`find_vma_intersection()` 函数返回第一个和指定地址区间相交的 VMA。因为该函数是内联函数，所以定义在文件 `<linux/mm.h>` 中：

```
static inline struct vm_area_struct *  
find_vma_intersection(struct mm_struct *mm,  
                     unsigned long start_addr,  
                     unsigned long end_addr)  
{  
    struct vm_area_struct *vma;  
  
    vma = find_vma(mm, start_addr);  
    if (vma && end_addr <= vma->vm_start)  
        vma = NULL;  
    return vma;  
}
```

第一个参数 `mm` 是要搜索的地址空间，`start_addr` 是区间的开始首位置，`end_addr` 是区间的尾位置。

显然，如果 `find_vma()` 返回 `NULL`，那么 `find_vma_intersection()` 也会返回 `NULL`。但是如果 `find_vma()` 返回有效的 VMA，`find_vma_intersection()` 只有在该 VMA 的起始位置于给定的地

址区间结束位置之前，才将其返回。如果 VMA 的起始位置大于指定地址范围的结束位置，则该函数返回 NULL。

15.5 mmap() 和 do_mmap()：创建地址区间

内核使用 do_mmap() 函数创建一个新的线性地址区间。但是说该函数创建了一个新 VMA 并不非常准确，因为如果创建的地址区间和一个已经存在的地址区间相邻，并且它们具有相同的访问权限的话，两个区间将合并为一个。如果不能合并，就确实需要创建一个新的 VMA 了。但无论哪种情况，do_mmap() 函数都会将一个地址区间加入到进程的地址空间中——无论是扩展已存在的内存区域还是创建一个新的区域。

do_mmap() 函数定义在文件 <linux/mm.h> 中。

```
unsigned long do_mmap(struct file *file, unsigned long addr,
                     unsigned long len, unsigned long prot,
                     unsigned long flag, unsigned long offset)
```

该函数映射由 file 指定的文件，具体映射的是文件中从偏移 offset 处开始，长度为 len 字节的范围内的数据。如果 file 参数是 NULL 并且 offset 参数也是 0，那么就代表这次映射没有和文件相关，该情况称作匿名映射 (anonymous mapping)。如果指定了文件名和偏移量，那么该映射称为文件映射 (file-backed mapping)。

addr 是可选参数，它指定搜索空闲区域的起始位置。

prot 参数指定内存区域中页面的访问权限。访问权限标志定义在文件 <asm/mman.h> 中，不同体系结构标志的定义有所不同，但是对所有体系结构而言，都会包含表 15-2 中所列举的标志。

表 15-2 页保护标志

标 志	对新建区间中页的要求
PROT_READ	对应于 VM_READ
PROT_WRITE	对应于 VM_WRITE
PROT_EXEC	对应于 VM_EXEC
PROT_NONE	页不可被访问

flag 参数指定了 VMA 标志，这些标志指定类型并改变映射的行为。它们也在文件 <asm/mman.h> 中定义，请参看表 15-3。

表 15-3 页保护标志

标 志	对新区间的要求
MAP_SHARED	映射可以被共享
MAP_PRIVATE	映射不能被共享
MAP_FIXED	新区间必须开始于指定的地址 addr
MAP_ANONYMOUS	映射不是 file-backed，而是匿名的
MAP_GROWSDOWN	对应于 VM_GROWSDOWN
MAP_DENYWRITE	对应于 VM_DENYWRITE

(续)

标 志	对新区间的要求
MAP_EXECUTABLE	对应于 VM_EXECUTABLE
MAP_LOCKED	对应于 VM_LOCKED
MAP_NORESERVE	不需要为映射保留空间
MAP_POPULATE	填充页表
MAP_NONBLOCK	在 I/O 操作上不堵塞

如果系统调用 `do_mmap()` 的参数中有无效参数, 那么它返回一个负值; 否则, 它会在虚拟内存中分配一个合适的新内存区域。如果有可能的话, 将新区域和邻近区域进行合并, 否则内核从 `vm_area_cachep` 长字节 (slab) 缓存中分配一个 `vm_area_struct` 结构体, 并且使用 `vma_link()` 函数将新分配的内存区域添加到地址空间的内存区域链表和红-黑树中, 随后还要更新内存描述符中的 `total_vm` 域, 然后才返回新分配的地址区间的初始地址。

在用户空间可以通过 `mmap()` 系统调用获取内核函数 `do_mmap()` 的功能。`mmap()` 系统调用定义如下:

```
void * mmap2(void *start,
             size_t length,
             int prot,
             int flags,
             int fd,
             off_t pgoff)
```

由于该系统调用是 `mmap()` 调用的第二种变种, 所以起名为 `mmap2()`。最原始的 `mmap()` 调用中最后一个参数是字节偏移量, 而目前这个 `mmap2()` 使用页面偏移作最后一个参数。使用页面偏移量可以映射更大的文件和更大的偏移位置。原始的 `mmap()` 调用由 POSIX 定义, 仍然在 C 库中作为 `mmap()` 方法使用, 但是内核中已经没有对应的实现了, 而实现的是新方法 `mmap2()`。虽然 C 库仍然可以使用原始版本的映射方法, 但是它其实还是基于函数 `mmap2()` 进行的, 因为对原始 `mmap()` 方法的调用是通过将字节偏移转化为页面偏移, 从而转化为对 `mmap2()` 函数的调用来实现的。

15.6 `mummap()` 和 `do_mummap()`: 删除地址区间

`do_mummap()` 函数从特定的进程地址空间中删除指定地址区间, 该函数定义在文件 `<linux/mm.h>` 中:

```
int do_mummap(struct mm_struct *mm, unsigned long start, size_t len)
```

第一个参数指定要删除区域所在的地址空间, 删除从地址 `start` 开始, 长度为 `len` 字节的地址区间。如果成功, 返回零。否则, 返回负的错误码。

系统调用 `mummap()` 给用户空间程序提供了一种从自身地址空间中删除指定地址区间的方法, 它和系统调用 `mmap()` 的作用相反:

```
int munmap(void *start, size_t length)
```

该系统调用定义在文件 `mm/mmap.c` 中，它是对 `do_mummap()` 函数的一个简单的封装：

```
asmlinkage long sys_munmap(unsigned long addr, size_t len)
{
    int ret;
    struct mm_struct *mm;

    mm = current->mm;
    down_write(&mm->mmap_sem);
    ret = do_munmap(mm, addr, len);
    up_write(&mm->mmap_sem);
    return ret;
}
```

15.7 页表

虽然应用程序操作的对象是映射到物理内存之上的虚拟内存，但是处理器直接操作的却是物理内存。所以当用程序访问一个虚拟地址时，首先必须将虚拟地址转化成物理地址，然后处理器才能解析地址访问请求。地址的转换工作需要通过查询页表才能完成，概括地讲，地址转换需要将虚拟地址分段，使每段虚拟地址都作为一个索引指向页表，而页表项则指向下一级别的页表或者指向最终的物理页面。

Linux 中使用三级页表完成地址转换。利用多级页表能够节约地址转换需占用的存放空间。如果利用三级页表转换地址，即使是 64 位机器，占用的空间也很有限。但是如果使用静态数组实现页表，那么即便在 32 位机器上，该数组也将占用巨大的存放空间。Linux 对所有体系结构，包括对那些不支持三级页表的体系结构（比如，有些体系结构只使用两级页表或者使用散列表完成地址转换）都使用三级页表管理，因为使用三级页表结构可以利用“最大公约数”的思想——一种设计简单的体系结构，可以按照需要在编译时简化使用页表的三级结构，比如只使用两级。

顶级页表是页全局目录（PGD），它包含了一个 `pgd_t` 类型数组，多数体系结构中 `pgd_t` 类型等同于无符号长整型类型。PGD 中的表项指向二级页目录中的表项：PMD。

二级页表是中间页目录（PMD），它是个 `pmd_t` 类型数组，其中的表项指向 PTE 中的表项。

最后一级的页表简称页表，其中包含了 `pte_t` 类型的页表项，该页表项指向物理页面。

多数体系结构中，搜索页表的工作是由硬件完成的（至少某种程度上）。虽然通常操作中，很多使用页表的工作都可以由硬件执行，但是只有在内核正确设置页表的前提下，硬件才能方便地操作它们。图 15-1 描述了虚拟地址通过页表找到物理地址的过程。

每个进程都有自己的页表（当然，线程会共享页表）。内存描述符的 `pgd` 域指向的就是进程的页全局目录。注意，操作和检索页表时必须使用 `page_table_lock` 锁，该锁在相应的进程的内存描述符中，以防止竞争条件。

页表对应的结构体依赖于具体的体系结构，所以定义在文件 `<asm/page.h>` 中。

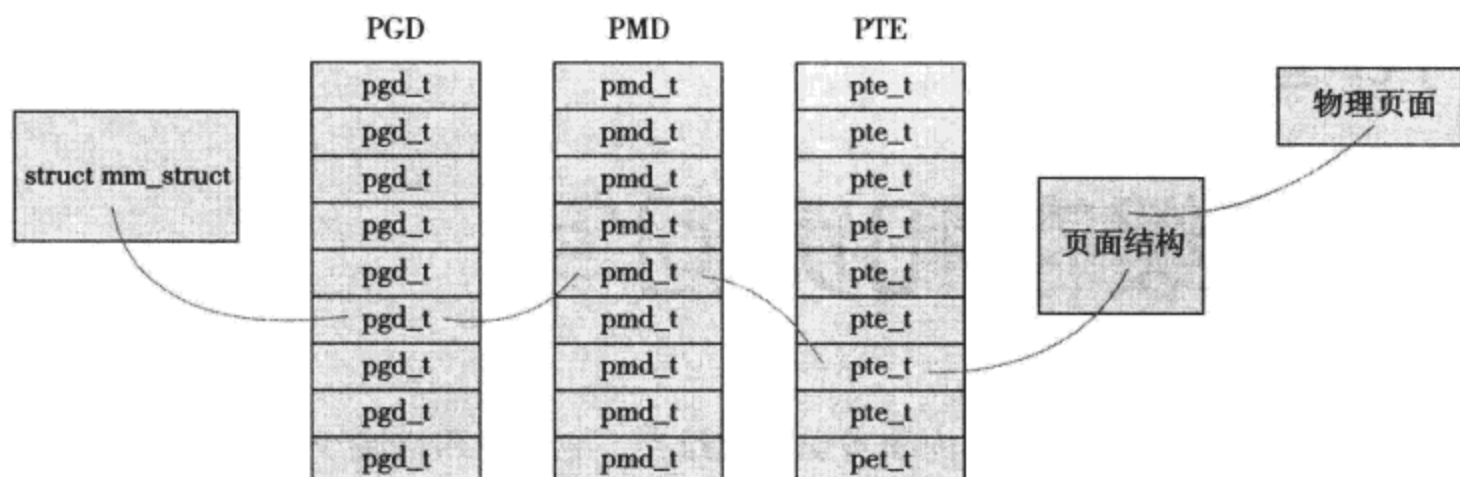


图 15-1 虚拟-物理地址查询

由于几乎每次对虚拟内存中的页面访问都必须先解析它，从而得到物理内存中的对应地址，所以页表操作的性能非常关键。但不幸的是，搜索内存中的物理地址速度很有限，因此为了加快搜索，多数体系结构都实现了一个翻译后备缓冲器（translate lookaside buffer, TLB）。TLB 作为一个将虚拟地址映射到物理地址的硬件缓存，当请求访问一个虚拟地址时，处理器将首先检查 TLB 中是否缓存了该虚拟地址到物理地址的映射，如果在缓存中直接命中，物理地址立刻返回；否则，就需要再通过页表搜索需要的物理地址。

虽然硬件完成了有关页表的部分工作，但是页表的管理仍然是内核的关键部分——而且在不断改进。2.6 版内核对页表管理的主要改进是：从高端内存分配部分页表。今后可能的改进包括通过在写时拷贝（copy-on-write）的方式共享页表。这种机制使得在 `fork()` 操作中可由父子进程共享页表。因为只有当子进程或父进程试图修改特定页表项时，内核才去创建该页表项的新拷贝，此后父子进程才不再共享该页表项。可以看到，利用共享页表可以消除 `fork()` 操作中页表拷贝所带来的消耗。

15.8 小结

这章的内容不能不说是很“难缠”啦。其中，我们看到了抽象出来的进程虚拟内存，看到了内核如何表示进程空间（通过 `mm_struct`）以及内核如何表示该空间中的内存区域（通过结构体 `vm_area_struct`）。除此以外，我们还了解了内核如何创建（通过 `mmap()`）和撤销（通过 `munmap()`）这些内存区域，最后还讨论了页表。因为 Linux 是一个基于虚拟内存的操作系统，所以这些概念对于系统运行来说都是非常基础的，一定要仔细领会。

第 16 章，我们要讨论页缓存——一种用于所有页 I/O 操作的内存数据缓存，而且还要涵盖内核执行基于页的数据回写。

第 16 章

页高速缓存和页回写

页高速缓存 (cache) 是 Linux 内核实现磁盘缓存。它主要用来减少对磁盘的 I/O 操作。具体地讲, 是通过把磁盘中的数据缓存到物理内存中, 把对磁盘的访问变为对物理内存的访问。这一章将讨论页高速缓存与页回写 (将页高速缓存中的变更数据刷新回磁盘的操作)。

磁盘高速缓存之所以在任何现代操作系统中尤为重要源自两个因素: 第一, 访问磁盘的速度要远远低于 (差好几个数量级) 访问内存的速度——ms 和 ns 的差距, 因此, 从内存访问数据比从磁盘访问速度更快, 若从处理器的 L1 和 L2 高速缓存访问则更快。第二, 数据一旦被访问, 就很有可能在短期内再次被访问到。这种在短时期内集中访问同一片数据的原理称作临时局部原理 (temporal locality)。临时局部原理能保证: 如果在第一次访问数据时缓存它, 那就极有可能在短期内再次被高速缓存命中 (访问到高速缓存中的数据)。正是由于内存访问要比磁盘访问快得多, 再加上数据一次被访问后更可能再次被访问的特点, 所以磁盘的内存缓存将给系统存储性能带来质的飞跃。

16.1 缓存手段

页高速缓存是由内存中的物理页面组成的, 其内容对应磁盘上的物理块。页高速缓存大小能动态调整——它可以通过占用空闲内存以扩张大小, 也可以自我收缩以缓解内存使用压力。我们称正被缓存的存储设备为后备存储, 因为缓存背后的磁盘无疑才是所有缓存数据的归属。当内核开始一个读操作 (比如, 进程发起一个 `read()` 系统调用), 它首先会检查需要的数据是否在页高速缓存中。如果在, 则放弃访问磁盘, 而直接从内存中读取。这个行为称作缓存命中。如果数据没有在缓存中, 称为缓存未命中, 那么内核必须调度块 I/O 操作从磁盘去读取数据。然后内核将读来的数据放入页缓存中, 于是任何后续相同的数据读取都可命中缓存了。注意, 系统并不一定要将整个文件都缓存。缓存可以持有某个文件的全部内容, 也可以存储另一些文件的一页或者几页。到底该缓存谁取决于谁被访问到。

16.1.1 写缓存

上面解释了在读操作过程中页高速缓存的作用, 那么在进程写磁盘时, 比如执行 `write()` 系统调用, 缓存如何被使用呢? 通常来讲, 缓存一般被实现成下面三种策略之一: 第一种策略称为不缓存 (nowrite), 也就是说高速缓存不去缓存任何写操作。当对一个缓存中的数据片进行写时, 将直接跳过缓存, 写到磁盘, 同时也使缓存中的数据失效。那么如果后续读操作进行时, 需要再重新从磁盘中读取数据。不过这种策略很少使用, 因为该策略不但不去缓存写操作, 而且需要额外费力去使缓存数据失效。

第二种策略，写操作将自动更新内存缓存，同时也更新磁盘文件。这种方式，通常称为写透缓存（write-through cache），因为写操作会立刻穿透缓存到磁盘中。这种策略对保持缓存一致性很有好处——缓存数据时刻和后备存储保持同步，所以不需要让缓存失效，同时它的实现也最简单。

第三种策略，也是 Linux 所采用的，称为“回写”^①。在这种策略下，程序执行写操作直接写到缓存中，后端存储不会立刻直接更新，而是将页高速缓存中被写入的页面标记成“脏”，并且被加入到脏页链表中。然后由一个进程（回写进程）周期行将脏页链表中的页写回到磁盘，从而让磁盘中的数据和内存中最终一致。最后清理“脏”页标识。注意这里“脏页”这个词可能引起混淆，因为实际上脏的并非页高速缓存中的数据（它们是干干净净的），而是磁盘中的数据（它们已过时了）。也许更好的描述应该是“未同步”吧。尽管如此，我们说缓存内容是脏的，而不是说磁盘内容。回写策略通常认为要好于写透策略，因为通过延迟写磁盘，方便在以后的时间内合并更多的数据和再一次刷新。当然，其代价是实现复杂度高了许多。

16.1.2 缓存回收

缓存算法最后涉及的重要内容是缓存中的数据如何清除；或者是为更重要的缓存项腾出位置；或者是收缩缓存大小，腾出内存给其他地方使用。这个工作，也就是决定缓存中什么内容将被清除的策略，称为缓存回收策略。Linux 的缓存回收是通过选择干净页（不脏）进行简单替换。如果缓存中没有足够的干净页面，内核将强制地进行回写操作，以腾出更多的干净可用页。最困难的事情在于决定什么页应该回收。理想的回收策略应该是回收那些以后最不可能使用的页面。当然要知道以后的事情你必须是先知。也正是这个原因，理想的回收策略称为预测算法。但这种策略太理想了，无法真正实现。

1. 最近最少使用

缓存回收策略通过所访问的数据特性，尽量追求预测效率。最成功的算法（特别是对于通用目的的页高速缓存）称作最近最少使用算法，简称 LRU。LRU 回收策略需要跟踪每个页面的访问踪迹（或者至少按照访问时间为序的页链表），以便能回收最老时间戳的页面（或者回收排序链表头所指的页面）。该策略的良好效果源自于缓存的数据越久未被访问，则越不大可能近期再被访问，而最近被访问的最有可能被再次访问。但是，LRU 策略并非是放之四海而皆准的法则，对于许多文件被访问一次，再不被访问的情景，LRU 尤其失败。将这些页面放在 LRU 链的顶端显然不是最优，当然，内核并没办法知道一个文件只会被访问一次，但是它却知道过去访问了多少次。

2. 双链策略

Linux 实现的是一个修改过的 LRU，也称为双链策略。和以前不同，Linux 维护的不再是一个 LUR 链表，而是维护两个链表：活跃链表和非活跃链表。处于活跃链表上的页面被认为是

^① 有些书或者操作系统称这个策略是 copy-write 或者 write-behind 缓存。所有这些命名都是同义词。Linux 和其他 Unix 系统使用 write-back 称谓来描述缓存策略，使用 writeback 称谓描述写缓存数据到后备存储这一动作。本书遵循这种称谓。

“热”的且不会被换出，而在非活跃链表上的页面则是可以被换出的。在活跃链表中的页面必须在其被访问时就处于非活跃链表中。两个链表都被伪 LRU 规则维护：页面从尾部加入，从头部移除，如同队列。两个链表需要维持平衡——如果活跃链表变得过多而超过了非活跃链表，那么活跃链表的头页面将被重新移回到非活跃链表中，以便能再被回收。双链表策略解决了传统 LRU 算法中对仅一次访问的窘境。而且也更加简单的实现了伪 LRU 语义。这种双链表方式也称作 LUR/2。更普遍的是 n 个链表，故称 LRU/ n 。

我们现在知道页缓存如何构建（通过读和写），如何在写时被同步（通过回写）以及旧数据如何被回收来容纳新数据（通过双链表）。现在让我们看看真实世界应用场景中，页高速缓存如何帮助系统。假定你在开发一个很大的软件工程（比如 Linux 内核）那么你将有大量的源文件被打开，只要你打开读取源文件，这些文件就将被存储在页高速缓存中。只要数据被缓存，那么从一个文件跳到另一个文件将瞬间完成。当你编辑文件时，存储文件也会瞬间完成，因为写操作只需要写到内存，而不是磁盘。当你编译项目时，缓存的文件将使得编译过程更少访问磁盘，所以编译速度也就更快了。如果整个源码树太大了，无法一次性放入内存，那么其中一部分必须被回收——由于双链表策略，任何回收的文件都将处于非活跃链表，而且不大可能是你正在编译的文件。幸运的是，在你没在编译的时候，内核会执行页回写，刷新你所修改文件的磁盘副本。由此可见，缓存将极大地提高系统性能。为了看到差别，对比一下缓存冷（cache cold）时（也就是说重启后，编译你的大软件工程的时间）和缓存热（cache warm）时的差别吧。

16.2 Linux 页高速缓存

从名字可以看出，页高速缓存缓存的是内存页面。缓存中的页来自对正规文件、块设备文件和内存映射文件的读写。如此一来，页高速缓存就包含了最近被访问过的文件的数据块。在执行一个 I/O 操作前（比如 `read()`^① 操作），内核会检查数据是否已经在页高速缓存中了，如果所需要的数据确实在高速缓存中，那么内核可以从内存中迅速地返回需要的页，而不再需要从相对较慢的磁盘上读取数据。在接下来的章节里，我们将剖析具体的数据结构以及内核如何使用它们管理缓存。

16.2.1 address_space 对象

在页高速缓存中的页可能包含了多个不连续的物理磁盘块^②。也正是由于页面中映射的磁盘块不一定连续，所以在页高速缓存中检查特定数据是否已经被缓存是件颇为困难的工作。因为不能用设备名称和块号来做页高速缓存中的数据的索引，要不然这将是最简单的定位办法。

另外，Linux 页高速缓存对被缓存的页面范围定义非常宽泛。实际上，在最初 System V

① 如你在第 13 章所见，并非 `read()`、`write()` 系统调用执行实际的页 I/O 操作，而是通过文件系统提供的特定操作 `file->f_op->read()` 和 `file->f_op->write()` 完成。

② 比如，x86 体系结构中一个物理页大小是 4KB。而大多数文件系统的块大小仅仅 512KB。所以八个块才可以填满一个页面。另外因为文件本身可能分布在磁盘上的各个位置，所以页面中映射的块也不需要连续。

Release 4 引入页高速缓存时，仅仅只用作缓存文件系统数据，所以 SVR4 的页高速缓存使用它的等价文件对象（称为 vnode 结构体）管理页高速缓存。Linux 页高速缓存的目标是缓存任何基于页的对象，这包含各种类型的文件和各种类型的内存映射。

虽然 Linux 页高速缓存可以通过扩展 inode 结构体（见第 13 章）支持页 I/O 操作，但这种做法会将页高速缓存局限于文件。为了维持页高速缓存的普遍性（不应该将其绑定到物理文件或者 inode 结构体），Linux 页高速缓存使用了一个新对象管理缓存项和页 I/O 操作。这个对象是 address_space 结构体。该结构体是第 15 章介绍的虚拟地址 vm_area_struct 的物理地址对等体。当一个文件可以被 10 个 vm_area_struct 结构体标识（比如有 5 个进程，每个调用 mmap() 映射它两次），那么这个文件只能有一个 address_space 数据结构——也就是文件可以有多个虚拟地址，但是只能在物理内存有一份。与 Linux 内核中其他结构一样，address_space 也是文不对题，也许更应该叫它 page_cache_entity 或者 physical_pages_of_a_file。

该结构定义在文件 <linux/fs.h> 中，下面给出具体形式：

```
struct address_space {
    struct inode          *host;                /* 拥有节点 */
    struct radix_tree_root page_tree;           /* 包含全部页面的 radix 树 */
    spinlock_t            tree_lock;            /* 保护 page_tree 的自旋锁 */
    unsigned int           i_mmap_writable;      /* VM_SHARED 计数 */
    struct prio_tree_root  i_mmap;              /* 私有映射链表 */
    struct list_head       i_mmap_nonlinear;    /* VM_NONLINEAR 链表 */
    spinlock_t            i_mmap_lock;         /* 保护 i_mmap 的自旋锁 */
    atomic_t               truncate_count;      /* 截断计数 */
    unsigned long          nrpages;             /* 页总数 */
    pgoff_t                writeback_index;     /* 回写的起始偏移 */
    struct address_space_operations *a_ops;     /* 操作表 */
    unsigned long          flags;               /* gfp_mask 掩码与错误标识 */
    struct backing_dev_info *backing_dev_info;  /* 预读信息 */
    spinlock_t            private_lock;        /* 私有 address_space 锁 */
    struct list_head       private_list;       /* 私有 address_space 链表 */
    struct address_space   *assoc_mapping;     /* 相关的缓冲 */
};
```

其中 i_mmap 字段是一个优先搜索树，它的搜索范围包含了在 address_space 中所有共享的与私有的映射页面。优先搜索树是一种巧妙地将堆与 radix 树结合^①的快速检索树。回忆早些提到的：一个被缓存的文件只和一个 address_space 结构体相关联，但它可以有多个 vm_area_struct 结构体——物理页到虚拟页是个一对多的映射。i_map 字段可帮助内核高效地找到关联的被缓存文件。

address_space 页总数由 nrpages 字段描述。

address_space 结构往往会和某些内核对象关联。通常情况下，它会与一个索引节点（inode）关联，这时 host 域就会指向该索引节点；如果关联对象不是一个索引节点的话，比如 address_space 和 swapper 关联时，host 域会被置为 NULL。

① 在内核中采用 Radix 优先搜索树是由 Edward M. McCreight 1985 年 5 月于 SIAM 计算机杂志第 14 期，第 2 集，257-276 页中提出的。

16.2.2 address_space 操作

a_ops 域指向地址空间对象中的操作函数表，这与 VFS 对象及其操作表关系类似，操作函数表定义在文件 <linux/fs.h> 中，由 address_space_operations 结构体来表示：

```
struct address_space_operations {
    int (*writepage)(struct page *, struct writeback_control *);
    int (*readpage)(struct file *, struct page *);
    int (*sync_page)(struct page *);
    int (*writepages)(struct address_space *,
                      struct writeback_control *);
    int (*set_page_dirty)(struct page *);
    int (*readpages)(struct file *, struct address_space *,
                     struct list_head *, unsigned);
    int (*write_begin)(struct file *, struct address_space *mapping,
                       loff_t pos, unsigned len, unsigned flags,
                       struct page **pagep, void **fsdata);
    int (*write_end)(struct file *, struct address_space *mapping,
                     loff_t pos, unsigned len, unsigned copied,
                     struct page *page, void *fsdata);
    sector_t (*bmap)(struct address_space *, sector_t);
    int (*invalidatepage)(struct page *, unsigned long);
    int (*releasepage)(struct page *, int);
    int (*direct_IO)(int, struct kiocb *, const struct iovec *,
                     loff_t, unsigned long);
    int (*get_xip_mem)(struct address_space *, pgoff_t, int,
                       void **, unsigned long *);
    int (*migratepage)(struct address_space *,
                       struct page *, struct page *);
    int (*launder_page)(struct page *);
    int (*is_partially_uptodate)(struct page *,
                                read_descriptor_t *,
                                unsigned long);
    int (*error_remove_page)(struct address_space *,
                             struct page *);
};
```

这些方法指针指向那些为指定缓存对象实现的页 I/O 操作。每个后备存储都通过自己的 address_space_operation 描述自己如何与页高速缓存交互。比如 ext3 文件系统在文件 fs/ext3/inode.c 中定义自己的操作表。这些方法提供了管理页高速缓存的各种行为，包括最常用的读页到缓存、更新缓存数据。这里面 readpage() 和 writepage() 两个方法最为重要。我们下面就来看看一个页面的读操作会包含哪些步骤。首先 linux 内核试图在页高速缓存中找到需要的数据；find_get_page() 方法负责完成这个检查动作。一个 address_space 对象和一个偏移量会传给 find_get_page() 方法，用于在页高速缓存中搜索需要的数据：

```
page = find_get_page(mapping, index);
```

这里 mapping 是指定的地址空间，index 是文件中的指定位置，以页面为单位（是的，称 address_space 结构体为 mapping，又是一个容易混淆的命名，虽然我也在重复这种内核命名不一

致的问题，但我还是鄙视它)。如果搜索的页并没在高速缓存中，`find_get_page()` 将会返回一个 `NULL`，并且内核将分配一个新页面，然后将之前搜索的页加入到页高速缓存中。

```
struct page *page;
int error;

/* 分配页... */
page = page_cache_alloc_cold(mapping);
if(!page)
    /* 内存分配出错 */

/*.....然后将其加入到页面调整缓存 */
error = add_to_page_cache_lru(page, mapping, index, GFP_KERNEL);
if(error)
    /* 页面被加入到页面高速缓存时，出错 */
```

最后，需要的数据从磁盘被读入，再被加入页高速缓存，然后返回给用户：

```
error= mapping->a_ops->readpage(file, page);
```

写操作和读操作有少许不同。对于文件映射来说，当页被修改了，VM 仅仅需要调用：

```
SetPageDirty(page);
```

内核会在晚些时候通过 `writepage()` 方法把页写出。对特定文件的写操作比较复杂，它的代码在文件 `mm/filemap.c` 中，通常写操作路径要包含以下各步：

```
page = __grab_cache_page(mapping, index, &cached_page, &lru_pvec);
status = a_ops->prepare_write(file, page, offset, offset+bytes);
page_fault = filemap_copy_from_user(page, offset, buf, bytes);
status = a_ops->commit_write(file, page, offset, offset+bytes);
```

首先，在页高速缓存中搜索需要的页。如果需要的页不在高速缓存中，那么内核在高速缓存中新分配一空闲项；下一步，内核创建一个写请求；接着数据被从用户空间拷贝到了内核缓冲；最后将数据写入磁盘。

因为所有的页 I/O 操作都要执行上面这些步骤，这就保证了所有的页 I/O 操作必然都是通过页高速缓存进行的。因此，内核也总是试图先通过页高速缓存来满足所有的读请求。如果在页高速缓存中未搜索到需要的页，则内核将从磁盘读入需要的页，然后将该页加入到页高速缓存中；对于写操作，页高速缓存更像是一个存储平台，所有要被写出的页都要加入页高速缓存中。

16.2.3 基树

因为在任何页 I/O 操作前内核都要检查页是否已经在页高速缓存中了，所以这种频繁进行的检查必须迅速、高效，否则搜索和检查页高速缓存的开销可能抵消页高速缓存带来的好处（至少在缓存命中率很低的时候，搜索的开销足以抵消以内存代替磁盘进行检索数据带来的好处）。

正如在 16.2.2 节所看到的，页高速缓存通过两个参数 `address_space` 对象加上一个偏移量进行搜索。每个 `address_space` 对象都有唯一的基树（radix tree），它保存在 `page_tree` 结构体中。基树是一个二叉树，只要指定了文件偏移量，就可以在基树中迅速检索到希望的页。页高速缓存的搜索函数 `find_get_page()` 要调用函数 `radix_tree_lookup()`，该函数会在指定基树中搜索指定页面。

基树核心代码的通用形式可以在文件 `lib/radix-tree.c` 中找到。另外，要想使用基树，需要包含头文件 `<linux/radix_tree.h>`。

16.2.4 以前的页散列表

在 2.6 版本以前，内核页高速缓存不是通过基树检索，而是通过一个维护了系统中所有页的全局散列表进行检索。对于给定的一个键值，该散列表会返回一个双向链表的入口对应于这个所给定的值。如果需要的页贮存在缓存中，那么链表中的一项就会与其对应。否则，页就不在页面高速缓存中，散列函数返回 `NULL`。

全局散列表主要存在四个问题：

- 由于使用单个的全局锁保护散列表，所以即使在中等规模的机器中，锁的争用情况也会相当严重，造成性能受损。
- 由于散列表需要包含所有页高速缓存中的页，可是搜索需要的只是和当前文件相关的那些页，所以散列表包含的页面相比搜索需要的页面要大得多。
- 如果散列搜索失败（也就是给定的页不在页高速缓存中），执行速度比希望的要慢得多，这是因为检索必须遍历指定散列键值对应的整个链表。
- 散列表比其他方法会消耗更多的内存。

2.6 版本内核中引入基于基树的页高速缓存来解决这些问题。

16.3 缓冲区高速缓存

独立的磁盘块通过块 I/O 缓冲也要被存入页高速缓存。回忆一下第 14 章，一个缓冲是一个物理磁盘块在内存里的表示。缓冲的作用就是映射内存中的页面到磁盘块，这样一来页高速缓存在块 I/O 操作时也减少了磁盘访问，因为它缓存磁盘块和减少块 I/O 操作。这个缓存通常称为缓冲区高速缓存，虽然实现上它没有作为独立缓存，而是作为页高速缓存的一部分。

块 I/O 操作一次操作一个单独的磁盘块。普遍的块 I/O 操作是读写 `i` 节点。内核提供了 `bread()` 函数实现从磁盘读一个块的底层操作。通过缓存，磁盘块映射到它们相关的内存页，并缓存到页高速缓存中。

缓冲和页高速缓存并非天生就是统一的，2.4 内核的主要工作之一就是统一它们。在更早的内核中，有两个独立的磁盘缓存：页高速缓存和缓冲区高速缓存。前者缓存页面，后者缓存缓冲区，这两个缓存并没有统一。一个磁盘块可以同时存于两个缓存中，这导致必须同步操作两个缓冲中的数据，而且浪费了内存，去存储重复的缓存项。今天我们只有一个磁盘缓存，即页高速缓存。虽然如此，内核仍然需要在内存中使用缓冲来表示磁盘块，幸好，缓冲是用页映射块的，所以它正好在页高速缓存中。

16.4 flusher 线程

由于页高速缓存的缓存作用，写操作实际上会被延迟。当页高速缓存中的数据比后台存储的数据更新时，该数据就称作脏数据。在内存中累积起来的脏页最终必须被写回磁盘。在以下 3 种情况发生时，脏页被写回磁盘：

- 当空闲内存低于一个特定的阈值时，内核必须将脏页写回磁盘以便释放内存，因为只有干净（不脏的）内存才可以被回收。当内存干净后，内核就可以从缓存清理数据，然后收缩缓存，最终释放出更多的内存。
- 当脏页在内存中驻留时间超过一个特定的阈值时，内核必须将超时的脏页写回磁盘，以确保脏页不会无限期地驻留在内存中。
- 当用户进程调用 `sync()` 和 `fsync()` 系统调用时，内核会按要求执行回写动作。

上面三种工作的目的完全不同。实际上，在旧内核中，这是由两个独立的内核线程（请看后面章节）分别完成的。但是在 2.6 内核中，由一群^①内核线程（flusher 线程）执行这三种工作。

首先，flusher 线程在系统中的空闲内存低于一个特定的阈值时，将脏页刷新写回磁盘。该后台回写例程的目的在于——在可用物理内存过低时，释放脏页以重新获得内存。这个特定的内存阈值可以通过 `dirty_background_ratio` sysctl 系统调用设置。当空闲内存比阈值 `dirty_background_ratio` 还低时，内核便会调用函数 `flusher_threads()`^②唤醒一个或多个 flusher 线程，随后 flusher 线程进一步调用函数 `bdi_writeback_all()` 开始将脏页写回磁盘。该函数需要一个参数——试图写回的页面数目。函数连续地写出数据，直到满足以下两个条件：

- 已经有指定的最小数目的页被写出到磁盘。
- 空闲内存数已经回升，超过了阈值 `dirty_background_ratio`。

上述条件确保了 flusher 线程操作可以减轻系统中内存不足的压力。回写操作不会在达到这两个条件前停止，除非刷新者线程写回了所有的脏页，没有剩下的脏页可再被写回了。

为了满足第二个目标，flusher 线程后台例程会被周期性唤醒（和空闲内存是否过低无关），将那些在内存中驻留时间过长的脏页写出，确保内存中不会有长期存在的脏页。如果系统发生崩溃，由于内存处于混乱之中，所以那些在内存中还没来得及写回磁盘的脏页就会丢失，所以周期性同步页高速缓存和磁盘非常重要。在系统启动时，内核初始化一个定时器，让它周期地唤醒 flusher 线程，随后使其运行函数 `wb_writeback()`。该函数将把所有驻留时间超过 `dirty_expire_interval` ms 的脏页写回。然后定时器将再次被初始化为 `dirty_expire_centisecs` 秒后唤醒 flusher 线程。总而言之，flusher 线程周期性地被唤醒并且把超过特定期限的脏页写回磁盘。

系统管理员可以在 `/proc/sys/vm` 中设置回写相关的参数，也可以通过 sysctl 系统调用设置它们。表 16-1 列出了与 `pdflush` 相关的所有可设置变量。

表 16-1 页回写设置

变 量	描 述
<code>dirty_background_ratio</code>	占全部内存的百分比。当内存中空闲页达到这个比例时， <code>pdflush</code> 线程开始回写脏页
<code>dirty_expire_interval</code>	该数值以百分之一秒为单位，它描述超时多久的数据将被周期性执行的 <code>pdflush</code> 线程写出
<code>dirty_ratio</code>	占全部内存百分比，当一个进程产生的脏页达到这个比例时，就开始被写出
<code>dirty_writeback_interval</code>	该数值以百分之一秒为单位，它描述 <code>pdflush</code> 线程的运行频率
<code>laptop_mode</code>	一个布尔值，用于控制膝上型计算机模式，具体请见后续内容

① 术语“群”通常在计算机科学中指的是一组可以并行执行的事情。

② 是的，它的确命名错了，它应该称为 `wakeup_bdflush()`。原因请看后面关于这个调用的继承部分。

flusher 线程的实现代码在文件 `mm/page-writeback.c` 和 `mm/backing-dev.c` 中，回写机制的实现代码在文件 `fs/fs-writeback.c` 中。

16.4.1 膝上型计算机模式

膝上型计算机模式是一种特殊的页回写策略，该策略主要意图是将硬盘转动的机械行为最小化，允许硬盘尽可能长时间地停滞，以此延长电池供电时间。该模式可通过 `/proc/sys/vm/laptop_mode` 文件进行配置。通常，上述配置文件内容为 0，也就是说膝上型计算机模式关闭，如果需要启用膝上型计算机模式，则向配置文件中写入 1。

膝上型计算机模式的页回写行为与传统方式相比只有一处变化。除了当缓存中的页面太旧时要执行回写脏页以外，flusher 还会找准磁盘运转的时机，把所有其他的物理磁盘 I/O、刷新脏缓冲等通通写回到磁盘，以便保证不会专门为了写磁盘而去主动激活磁盘运行。

上述回写行为变化要求 `dirty_expire_interval` 和 `dirty_writeback_interval` 两阈值必须设置得更大，比如 10 分钟。因为磁盘运转并不很频繁，所以用这样长的回写延迟就能保证膝上型计算机模式可以等到磁盘运转机会写入数据。因为关闭磁盘驱动器是节电的重要手段，膝上模式可以延长膝上计算机依靠电池的续航能力。其坏处则是系统崩溃或者其他错误会使得数据丢失。

多数 Linux 发布版会在计算机接上电池或拔掉电池时，自动开启或禁止膝上型计算机模式以及其他需要的回写可调节开关。因此机器可在使用电池电源时自动进入膝上型计算机模式，而在插上交流电源时恢复到常规的页回写模式。

16.4.2 历史上的 bdflush、kupdated 和 pdflush

在 2.6 版本前，flusher 线程的工作是分别由 bdflush 和 kupdated 两个线程共同完成。

当可用内存过低时，bdflush 内核线程在后台执行脏页回写操作。类似 flusher，它也是一组阈值参数，当系统中空闲内存消耗到特定阈值以下时，bdflush 线程就被 `wakeup_bdflush()` 函数唤醒。

bdflush 和当前的 flusher 线程之间存在两个主要区别。第一个区别是系统中只有一个 bdflush 后台线程，而 flusher 线程的数目却是根据磁盘数量变化的（这在 16.5 节中会谈到）；第二个区别是 bdflush 线程基于缓冲，它将脏缓冲写回磁盘。相反，flusher 线程基于页面，它将整个脏页写回磁盘。当然，页面可能包含缓冲，但是实际 I/O 操作对象是整页，而不是块。因为页在内存中是更普遍和普通的概念，所以管理页相比管理块要简单。

因为只有在内存过低和缓冲数量过大时，bdflush 例程才刷新缓冲，所以 kupdated 例程被引入，以便周期地写回脏页。它和 pdflush 线程的 `wb_writeback()` 函数提供同样的服务。

在 2.6 内核中，bdflush 和 kupdated 已让路给了 pdflush 线程——page dirty flush（比以前两个更容易令人混淆的名字）的缩写。Pdflush 线程的执行和今天的 flusher 线程类似。其主要区别在于，pdflush 线程数目是动态的，默认是 2 个到 8 个，具体多少取决于系统 I/O 的负载。Pdflush 线程与任何任务都无关，它们是面向系统所有磁盘的全局任务。这样做的好处是实现简单，可带来的问题是，pdflush 线程很容易在拥塞的磁盘上绊住，而现代硬件发生拥塞更是家常便饭。采用每个磁盘一个刷新线程可以使得 I/O 操作同步执行，简化了拥塞逻辑，也提升

了性能。Flusher 线程在 2.6.32 内核系列中取代了 pdflush 线程（针对每个磁盘独立执行回写操作是其和 pdflush 的主要区别）。本节中剩下部分的讨论，仍然适用于 pdflush，而且也适用于所有 2.6 内核系列。

16.4.3 避免拥塞的方法：使用多线程

使用 bdfush 线程最主要的一个缺点就是，bdfush 仅仅包含了一个线程，因此很有可能在页回写任务很重时，造成拥塞。这是因为单一的线程有可能堵塞在某个设备的已拥塞请求队列（正在等待将请求提交给磁盘的 I/O 请求队列）上，而其他设备的请求队列却没法得到处理。如果系统有多个磁盘和较强的处理能力，内核应该能使得每个磁盘都处于忙状态。不幸的是，即使还有许多数据需要回写，单个的 bdfush 线程也可能会堵塞在某个队列的处理上，不能使所有磁盘都处于饱和的工作状态，原因在于磁盘的吞吐量是非常有限的。正是因为磁盘的吞吐量很有限，所以如果只有唯一线程执行页回写操作，那么这个线程很容易苦苦等待对一个磁盘上的操作。为了避免出现这种情况，内核需要多个回写线程并发执行，这样单个设备队列的拥塞就不会成为系统瓶颈了。

2.6 内核通过使用多个 flusher 线程来解决上述问题。每个线程可以相互独立地将脏页刷新回磁盘，而且不同的 flusher 线程处理不同的设备队列。pdflush 线程策略中，线程数是动态变化的。每一个线程试图尽可能忙地从每个超级块的脏页链表中回收数据，并且写回到磁盘。pdflush 方式避免了因为一个忙磁盘，而使得其余磁盘饥饿的状况。通常情况下这样是不错的，但是如果每个 pdflush 线程在同一个拥塞的队列上挂起了又该如何呢？在这种情况下，多个 pdflush 线程可能并不比一个线程更好，就浪费的内存而言就要多许多。为了减轻上述影响，pdflush 线程采用了拥塞回避策略：它们会主动尝试从那些没有拥塞的队列回写页。从而，pdflush 线程将其工作调度开来，防止了仅仅欺负某一个忙碌设备。

这种方式效果确实不错，但是拥塞回避并不完美。在现代操作系统中，因为 I/O 总线技术和计算机其他部分相比发展要缓慢得多，所以拥塞现象时常发生——处理器发展速度遵循摩尔定理，但是硬盘驱动器则仅仅比 20 年前快一点点。要知道，目前除了 pdflush 以外，I/O 系统中还没有其他地方使用这种拥塞回避处理。不过在很多情况下，pdflush 确实可以避免向特定盘回写的时间和期望时间相比太久。当前 flusher 线程模型（自 2.6.32 内核系列以后采用）和具体块设备关联，所以每个给定线程从每个给定设备的脏页链表收集数据，并写回到对应磁盘。回写于是更趋于同步了，而且由于每个磁盘对应一个线程，所以线程也不需要采用复杂的拥塞避免策略，因为一个磁盘就一个线程操作。该方法提高了 I/O 操作的公平性，而且降低了饥饿风险。

因为使用 pdflush 以及后来的 flusher 线程提升了页回写性能。2.6 内核系列相比早期内核可以让磁盘利用更饱和。在系统 I/O 很重的时候，flusher 线程可以在每个磁盘上都维护更高的吞吐量。

16.5 小结

本章中我们看到了 Linux 的页高速缓存和页回写。了解了内核如何通过页缓存执行页 I/O 操作以及这些页高速缓存（通过存储数据在内存中）可以利用减少磁盘 I/O，从而极大地提升系统

的性能。我们讨论了通过称为“回写缓存”的进程维护在缓存中的更新页面——具体做法是标记内存中的页面为脏，然后找时机延迟写到磁盘中。Flusher 内核线程将负责处理这些最终的页回写操作。

通过最近几章的学习，你应该已经对内存与文件系统有了深刻认识，那么接下来我们将进入模块专题，去学习 Linux 的设备驱动以及内核如何被模块化、在运行时插入和删除内核代码的动态机制。



第 17 章

设备与模块

在本章中，关于设备驱动和设备管理，我们讨论四种内核成分。

- 设备类型：在所有 Unix 系统中为了统一普通设备的操作所采用的分类。
- 模块：Linux 内核中用于按需加载和卸载目标码的机制。
- 内核对象：内核数据结构中支持面向对象的简单操作，还支持维护对象之间的父子关系。
- sysfs：表示系统中设备树的一个文件系统。

17.1 设备类型

在 Linux 以及所有 Unix 系统中，设备被分为以下三种类型：

- 块设备
- 字符设备
- 网络设备

块设备通常缩写为 blkdev，它是可寻址的，寻址以块为单位，块大小随设备不同而不同；块设备通常支持重定位（seeking）操作，也就是对数据的随机访问。块设备的例子有硬盘、蓝光光碟，还有如 Flash 这样的存储设备。块设备是通过称为“块设备节点”的特殊文件来访问的，并且通常被挂载为文件系统。我们在第 13 章已经讨论过了文件系统，在第 14 章已经讨论过了块设备。

字符设备通常缩写为 cdev，它是不可寻址的，仅提供数据的流式访问，就是一个个字符，或者一个个字节。字符设备的例子有键盘、鼠标、打印机，还有大部分伪设备。字符设备是通过称为“字符设备节点”的特殊文件来访问的。与块设备不同，应用程序通过直接访问设备节点与字符设备交互。

网络设备最常见的类型有时也以以太网设备（ethernet devices）来称呼，它提供了对网络（例如 Internet）的访问，这是通过一个物理适配器（如你的膝上型计算机的 802.11 卡）和一种特定的协议（如 IP 协议）进行的。网络设备打破了 Unix 的“所有东西都是文件”的设计原则，它不是通过设备节点来访问，而是通过套接字 API 这样的特殊接口来访问。

Linux 还提供了不少其他设备类型，但都是针对单个任务，而非通用的。一个特例是“杂项设备”（miscellaneous device），通常简写为 miscdev，它实际上是个简化的字符设备。杂项设备使驱动程序开发者能够很容易地表示一个简单设备——实际上是对通用基本架构的一种折中。

并不是所有设备驱动都表示物理设备。有些设备驱动是虚拟的，仅提供访问内核功能而已。我们称为“伪设备”（pseudo device），最常见的如内核随机数发生器（通过 /dev/random 和 /dev/urandom 访问）、空设备（通过 /dev/null 访问）、零设备（通过 /dev/zero 访问）、满设备

(通过 /dev/full 访问), 还有内存设备 (通过 /dev/mem 访问)。然而, 大部分设备驱动是表示物理设备的。

17.2 模块

尽管 Linux 是“单块内核”(monolithic)的操作系统——这是说整个系统内核都运行于一个单独的保护域中, 但是 Linux 内核是模块化组成的, 它允许内核在运行时动态地向其中插入或从中删除代码。这些代码 (包括相关的子例程、数据、函数入口和函数出口) 被一并组合在一个单独的二进制镜像中, 即所谓的可装载内核模块中, 或简称为模块。支持模块的好处是基本内核镜像可以尽可能地小, 因为可选的功能和驱动程序可以利用模块形式再提供。模块允许我们方便地删除和重新载入内核代码, 也方便了调试工作。而且当热插拔新设备时, 可通过命令载入新的驱动程序。

本章我们将探寻内核模块的奥秘, 同时也学习如何编写自己的内核模块。

17.2.1 Hello, World

与开发我们已经讨论过的大多数内核核心子系统不同, 模块开发更接近编写新的应用系统, 因为至少在模块文件中具有入口和出口点。

虽然编写“Hello, World”程序作为实例实属陈词滥调了, 但它的确很让人喜爱。内核模块 Hello, World 出场了:

```
/*
 * hello.c _The Hello, World! 我们的第一个内核模块
 */

#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
/*
 * hello_init- 初始化函数, 当模块装载时被调用, 如果成功装载返回零, 否
 * 则返回非零值
 */
static int hello_init(void)
{
    printk(KERN_ALERT "I bear a charmed life.\n");
    return 0;
}

/*
 * hello_exit- 退出函数, 当模块卸载时被调用
 */
static void hello_exit(void)
{
    printk(KERN_ALERT "Out, out, brief candle!\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

```
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("Shakespeare");  
MODULE_DESCRIPTION("A Hello, World Module");
```

这大概是我们所能见到的最简单的内核模块了，`hello_init()` 函数是模块的入口点，它通过 `module_init()` 例程注册到系统中，在内核装载时被调用。调用 `module_init()` 实际上不是真正的函数调用，而是一个宏调用，它唯一的参数便是模块的初始化函数。模块的所有初始化函数必须符合下面的形式：

```
int my_init(void);
```

因为 `init` 函数通常不会被外部函数直接调用，所以你不必导出该函数，故它可标记为 `static` 类型。

`init` 函数会返回一个 `int` 型数值，如果初始化（或你的 `init` 函数想做的事情）顺利完成，那么它的返回值为零；否则返回一个非零值。

这个 `init` 函数仅仅打印了一条简单的消息，然后返回零。在实际的模块中，`init` 函数还会注册资源、初始化硬件、分配数据结构等。如果这个文件被静态编译进内核映像中，其 `init` 函数将存放在内核映像中，并在内核启动时运行。

`hello_exit()` 函数是模块的出口函数，它由 `module_exit()` 例程注册到系统。在模块从内存卸载时，内核便会调用 `hello_exit()`。退出函数可能会在返回前负责清理资源，以保证硬件处于一致状态；或者做其他的一些操作。简单说来，`exit` 函数负责对 `init` 函数以及在模块生命周期过程中所做的一切事情进行撤销工作，基本上就是清理工作。在退出函数返回后，模块就被卸载了。

退出函数必须符合以下形式：

```
void my_exit(void);
```

与 `init` 函数一样，你也可以标记其为 `static`。

如果上述文件被静态地编译到内核映像中，那么退出函数将不被包含，而且永远都不会被调用（因为如果不是编译成模块的话，那么代码就不需从内核中卸载）。

`MODULE_LICENSE()` 宏用于指定模块的版权。如果载入非 GPL 模块到系统内存，则会在内核中设置被污染标识——这个标识只起到记录信息的作用。版权许可证具有两大目的。首先，它具有通告的目的。当 `oops` 中设置了被污染的标识时，很多内核开发者对 `bug` 的报告缺乏信任，因为他们认为二进制模块（也就是开发者不能调试它）被装载到了内核。其次，非 GPL 模块不能调用 `GPL_only` 符号，本章后续的“导出符号表”一节将对其加以描述。

最后还要说明，`MODULE_AUTHOR()` 宏和 `MODULE_DESCRIPTION()` 宏指定了代码作者和模块的简要描述，它们完全是用作信息记录目的。

17.2.2 构建模块

在 2.6 内核中，由于采用了新的“`kbuild`”构建系统，现在构建模块相比从前更加容易。构建过程的第一步是决定在哪里管理模块源码。你可以把模块源码加入到内核源代码树中，或者是作为一个补丁或者是最终把你的代码合并到正式的内核代码树中；另一种可行的方式是在内核源

代码树之外维护和构建你的模块源码。

1. 放在内核源代码树中

最理想的情况莫过于你的模块正式成为 Linux 内核的一部分，这样就会被存放入内核源代码树中。把你的模块代码正确地置于内核中，开始的时候难免需要更多的维护，但这样通常是一劳永逸的解决之道。

当你决定了把你的模块放入内核源代码树中，下一步要清楚你的模块应在内核源代码树中处于何处。设备驱动程序存放在内核源码树根目录下 `/drivers` 的子目录下，在其内部，设备驱动文件被进一步按照类别、类型或特殊驱动程序等更有序地组织起来。如字符设备存在于 `drivers/char/` 目录下，而块设备存放在 `drivers/block/` 目录下，USB 设备则存放在 `drivers/usb/` 目录下。文件的具体组织规则并不须绝对墨守成规，不容打破，你可看到许多 USB 设备也属于字符设备。但是不管怎么样，这些组织关系对我们来说相当容易理解，而且很也准确。

假定你有一个字符设备，而且希望将它存放在 `drivers/char/` 目录下，那么要注意，在该目录下同时会存在大量的 C 源代码文件和许多其他目录。所以对于仅仅只有一两个源文件的设备驱动程序，可以直接存放在该目录下；但如果驱动程序包含许多源文件和其他辅助文件，那么可以创建一个新子目录。这期间并没有什么金科玉律。假设想建立自己代码的子目录，你的驱动程序是一个钓鱼竿和计算机的接口，名为 Fish Master XL 3000，那么你需要在 `drivers/char/` 目录下建立一个名为 `fishing` 的子目录。

接下来需要向 `drivers/char/` 下的 Makefile 文件中添加一行。编辑 `drivers/char/Makefile/` 并加入：

```
obj-m += fishing/
```

这行编译指令告诉模块构建系统，在编译模块时需要进入 `fishing/` 子目录中。更可能发生的情况是，你的驱动程序的编译取决于一个特殊配置选项；比如，可能的 `CONFIG_FISHING_POLE`（请看 17.2.6 节，它会告诉你如何加入一个新的编译选项）。如果这样，你需要用下面的指令代替刚才那条指令：

```
obj-$(CONFIG_FISHING_POLE) += fishing/
```

最后，在 `drivers/char/fishing/` 下，需要添加一个新 Makefile 文件，其中需要有下面这行指令：

```
obj-m += fishing.o
```

一切就绪了，此刻构建系统运行将会进入 `fishing/` 目录下，并且将 `fishing.c` 编译为 `fishing.ko` 模块。虽然你写的扩展名是 `.o`，但是模块被编译后的扩展名却是 `.ko`。

再一个可能，要是你的钓鱼竿驱动程序编译时有编译选项，那么你可能需要这么来做：

```
obj-$(CONFIG_FISHING_POLE) += fishing.o
```

以后，假如你的钓鱼竿驱动程序需要更加智能化——它可以自动检测钓鱼线，这可是最新的鱼竿“必备要求”呀。这时驱动程序源文件可能就不再只有一个了。别怕，朋友，你只要把你的 Makefile 做如下修改就可搞定：

```
obj-$(CONFIG_FISHING_POLE) += fishing.o
fishing-objs := fishing-main.o fishing-line.o
```

每当设置了 `CONFIG_FISHING_POLE`, `fishing-main.c` 和 `fishing-line.c` 就一起被编译和连接到 `fishing.ko` 模块内。

最后一个注意事项是, 在构建文件时你可能需要额外的编译标记, 如果这样, 你只需在 `Makefile` 中添加如下指令:

```
EXTRA_CFLAGS += -DTITANIUM_POLE
```

如果喜欢把你的源文件置于 `drivers/char/` 目录下, 并且不建立新目录的话, 那么你要做的便是将前面提到的行 (也就是原来处于 `drivers/char/fishing/` 下你自己的 `Makefile` 中的) 都加入到 `drivers/char/Makefile` 中。

开始编译吧, 运行内核构建过程和原来一样。如果你的模块编译取决于配置选项, 比如有 `CONFIG_FISHING_POLE` 约束, 那么在编译前首先要确保选项被允许。

2. 放在内核代码外

如果你喜欢脱离内核源代码树来维护和构建你的模块, 把自己作为一个圈外人, 那你要做的就是在你自己的源代码树目录中建立一个 `Makefile` 文件, 它只需要一行指令:

```
obj-m := fishing.o
```

这条指令就可把 `fishing.c` 编译成 `fishing.ko`。如果你有多个源文件, 那么用两行就足够:

```
obj-m := fishing.o
fishing-objs := fishing-main.o fishing-line.o
```

这样一来, `fishing-main.c` 和 `fishing-line.c` 就一起被编译和连接到 `fishing.ko` 模块内了。

模块在内核内和在内核外构建的最大区别在于构建过程。当模块在内核源代码树外围时, 你必须告诉 `make` 如何找到内核源代码文件和基础 `Makefile` 文件。不过要完成这个工作同样不难:

```
make -C /kernel/source/location SUBDIRS=$PWD modules
```

在这个例子中, `/kernel/source/location` 是你配置的内核源代码树。回想一下, 不要把要处理的内核源代码树放在 `/usr/src/linux` 下, 而要移到你 `home` 目录下某个方便访问的地方。

17.2.3 安装模块

编译后的模块将被装入到目录 `/lib/modules/version/kernel/` 下, 在 `kernel/` 目录下的每一个目录都对应着内核源码树中的模块位置。如果使用的是 2.6.34 内核, 而且将你的模块源代码直接放在 `drivers/char/` 下, 那么编译后的钓鱼竿驱动程序的存放路径将是: `/lib/modules/2.6.34/kernel/drivers/char/fishing.ko`。

下面的构建命令用来安装编译的模块到合适的目录下:

```
make modules_install
```

通常需要以 `root` 权限运行。

17.2.4 产生模块依赖性

Linux 模块之间存在依赖性, 也就是说钓鱼模块依赖于鱼饵模块, 那么当你载入钓鱼模块

时，鱼饵模块会被自动载入。这里需要的依赖信息必须事先生成。多数 Linux 发布版都能自动产生这些依赖关系信息，而且在每次启动时更新。若想产生内核依赖关系的信息，root 用户可运行命令

```
depmod
```

为了执行更快的更新操作，那么可以只为新模块生成依赖信息，而不是生成所有的依赖关系，这时 root 用户可运行命令

```
depmod -A
```

模块依赖关系信息存放在 `/lib/modules/version/modules.dep` 文件中。

17.2.5 载入模块

载入模块最简单的方法是通过 `insmod` 命令，这是个功能很有限的命令，它能做的就是请求内核载入指定的模块。`insmod` 程序不执行任何依赖性分析或进一步的错误检查。它用法简单，以 root 身份运行命令：

```
insmod module.ko
```

这里，`module.ko` 是要载入的模块名称。比如装载钓鱼竿模块，那你就执行命令：

```
insmod fishing.ko
```

类似的，卸载一个模块，你可使用 `rmmod` 命令，它同样需要以 root 身份运行：

```
rmmod module
```

比如，`rmmod fishing` 命令将卸载钓鱼竿模块。

```
rmmod fishing
```

这两个命令是很简单，但是它们一点也不智能。先进工具 `modprobe` 提供了模块依赖性分析、错误智能检查、错误报告以及许多其他功能和选项。我强烈建议大家用这个命令。

为了在内核 via `modprobe` 中插入模块，需要以 root 身份运行：

```
modprobe module [ module parameters ]
```

其中，参数 `module` 指定了需要载入的模块名称，后面的参数将在模块加载时传入内核。（请看 17.2.7 一节对模块参数的讨论）。

`modprobe` 命令不但会加载指定的模块，而且会自动加载任何它所依赖的有关模块。所以说它是加载模块的最佳机制。

`modprobe` 命令也可用来从内核中卸载模块，当然这也需要以 root 身份运行：

```
modprobe -r modules
```

参数 `modules` 指定一个或多个需要卸载的模块。与 `rmmod` 命令不同，`modprobe` 也会卸载给定模块所依赖的相关模块，但其前提是这些相关模块没有被使用。Linux 用户手册第 8 部分提供

了上述命令的使用参考，里面包括了命令选项和用法。

17.2.6 管理配置选项

在前面的内容中我们看到，只要设置了 CONFIG_FISHING_POLE 配置选项，钓鱼竿模块将被自动编译。虽然配置选项在前面已经讨论过了，但这里我们将继续以钓鱼竿驱动程序为例，再看看一个新的配置选项如何加入。

由于 2.6 内核中新引入了“kbuild”系统，因此，加入一个新配置选项现在可以说是易如反掌。你所需做的全部就是向 kconfig 文件中添加一项，用以对应内核源码树。对驱动程序而言，kconfig 通常和源代码处于同一目录。如果钓鱼竿驱动程序在目录 drivers/char/ 下，那么你便会发现 drivers/char/kconfig 也同时存在。

如果你建立了一个新子目录，而且也希望 kconfig 文件存在于该目录中的话，那么你必须在一个已存在的 kconfig 文件中将它引入。你需要加入下面一行指令：

```
source "drivers/char/fishing/Kconfig"
```

这里所谓存在的 Kconfig 文件可能是 drivers/char/Kconfig。

Kconfig 文件很方便加入一个配置选型，请看钓鱼竿模块的选项，如下所示：

```
config FISHING_POLE
    tristate "Fish Master 3000 support"
    default n
    help
        If you say Y here, support for the Fish Master 3000 with computer
        interface will be compiled into the kernel and accessible via a
        device node. You can also say M here and the driver will be built as a
        module named fishing.ko.

    If unsure, say N.
```

配置选项第一行定义了该选项所代表的配置目标。注意 CONFIG_ 前缀并不需要写上。

第二行声明选项类型为 tristate，也就是说可以编译进内核（Y），也可作为模块编译（M），或者干脆不编译它（N）。如果编译选项代表的是一个系统功能，而不是一个模块，那么编译选项将用 bool 指令代替 tristate，这说明它不允许被编译成模块。处于指令之后的引号内文字为该选项指定了名称。

第三行指定了该选项的默认选择，这里默认操作是不编译它（N）。也可以把默认选择指定为编译进内核（Y），或者编译成一个模块（M）。对驱动程序而言，默认选择通常为不编译进内核（N）。

Help 指令的目的是为该选项提供帮助文档。各种配置工具都可以按要求显示这些帮助。因为这些帮助是面向编译内核的用户和开发者的，所以帮助内容简洁扼要。一般的用户通常不会编译内核，但如果他们想试试，往往也能理解配置帮助的意思。

除了上述选项以外，还存在其他选项。比如 depends 指令规定了在该选项被设置前，首先要设置的选项。假如依赖性不满足，那么该选项就被禁止。比如，如果你加入指令：

```
depends on FISH_TANK
```

到配置选项中，那么就意味着在 CONFIG_FISH_TANK 被选择前，我们的钓鱼竿模块是不能使用的（Y 或者 M）。Select 指令和 depends 类似，它们只有一点不同之处——只要是 select 指定了谁，它就会强行将被指定的选项打开。所以这个指令可不能向 depends 那样滥用一通，因为它会自动的激活其他配置选项。它的用法和 depends 一样。比如：

```
select BAIT
```

意味着当 CONFIG_FISHING_POLE 被激活时，配置选项 CONFIG_BAIT 必然一起被激活。

如果 select 和 depends 同时指定多个选项，那就需要通过 && 指令来进行多选。使用 depends 时，你还可以利用叹号前缀来指明禁止某个选项。比如：

```
depends on EXAMPLE_DRIVERS && !NO_FISHING_ALLOWED
```

这行指令就指定驱动程序安装要求打开 CONFIG_EXAMPLE_DRIVERS 选项，同时要禁止 CONFIG_NO_FISHING_ALLOWED 选项。

tristate 和 bool 选项往往会结合 if 指令一起使用，这表示某个选项取决于另一个配置选项。如果条件不满足，配置选项不但会被禁止，甚至不会显示在配置工具中，比如，要求配置系统只有在 CONFIG_x86 配置选项设置时才显示某选项。请看下面指令：

```
bool "Deep Sea Mode" if OCEAN
```

If 指令也可与 default 指令结合使用，强制只有在条件满足时 default 选项才有效。

配置系统导出了一些元选项（meta-option）以简化生成配置文件。比如选项 CONFIG_EMBEDDED 是用于关闭那些用户想要禁止的关键功能（比如要在嵌入系统中节省珍贵的内存）；选项 CONFIG_BROKEN_ON_SMP 用来表示驱动程序并非多处理器安全的。通常该项不应设置，标记它的目的是确保用户能知道该驱动程序的弱点。当然，新的驱动程序不应该使用该标志。最后要说明 CONFIG_EXPERIMENTAL 选项，它是一个用于说明某项功能尚在试验或处于 beta 阶段的标志选项。该选项默认情况下关闭，同样，标记它的目的是为了用户在使用驱动程序前明白潜在风险。

17.2.7 模块参数

Linux 提供了这样一个简单框架——它可允许驱动程序声明参数，从而用户可以在系统启动或者模块装载时再指定参数值，这些参数对于驱动程序属于全局变量。值得一提的是模块参数同时也将出现在 sysfs 文件系统中（见本章后面的介绍），这样一来，无论是生成模块参数，还是管理模块参数的方式都变得灵活多样了。

定义一个模块参数可通过宏 module_param() 完成：

```
module_param(name, type, perm);
```

参数 name 既是用户可见的参数名，也是你模块中存放模块参数的变量名。参数 type 则存放了参数的类型，它可以是 byte、short、ushort、int、uint、long、ulong、charp、bool 或 invbool，它们分别代表字节型、短整型、无符号短整形、整型、无符号整型、长整形、无符号长整型、字

符指针、布尔型，以及应用户要求转换得来的布尔型。其中 `byte` 类型存放在 `char` 类型变量中，`boolean` 类型存放在 `int` 变量中，其余的类型都一致对应 C 语言的变量类型。最后一个参数 `perm` 指定了模块在 `sysfs` 文件系统下对应文件的权限，该值可以是八进制的格式，比如 `0644`（所有者可以读写，组内可以读，其他人可以读）；或是 `S_Ifoo` 的定义形式，比如 `S_IRUGO | S_IWUSR`（任何人可读，`user` 可写）；如果该值是零，则表示禁止所有的 `sysfs` 项。

上面的宏其实并没有定义变量，你必须在使用该宏前进行变量定义。通常使用类似下面的语句完成定义：

```
/* 在模块参数控制下，我们允许在钓鱼竿上用活鱼饵 */
static int allow_live_bait = 1;          /* 默认功能允许 */
module_param(allow_live_bait, bool, 0644); /* 一个 Boolean 类型 */
```

这个值处于模块代码文件外部，换句话说，`allow_live_bait` 是个全局变量。

有可能模块的外部参数名称不同于它对应的内部变量名称，这时就该使用宏 `module_param_named()` 定义了：

```
module_param_named(name, variable, type, perm);
```

参数 `name` 是外部可见的参数名称，参数 `variable` 是参数对应的内部全局变量名称。比如：

```
static unsigned int max_test = DEFAULT_MAX_LINE_TEST;
module_param_named(maximum_line_test, max_test, int, 0);
```

通常，需要用 `charp` 类型来定义模块参数（一个字符串），内核将用户提供的这个字符串拷贝到内存，而且将变量指向该字符串。比如：

```
static char *name;
module_param(name, charp, 0);
```

如果需要，也可使内核直接拷贝字符串到指定的字符数组。宏 `module_param_string()` 可完成上述任务：

```
module_param_string(name, string, len, perm);
```

这里参数 `name` 为外部参数名称，参数 `string` 是对应的内部变量名称，参数 `len` 是 `string` 命名缓冲区的长度（或更小的长度，但是没什么太大的意义），参数 `perm` 是 `sysfs` 文件系统的访问权限（如果为零，则表示完全禁止 `sysfs` 项），比如：

```
static char species[BUF_LEN];
module_param_string(specifies, species, BUF_LEN, 0);
```

你可接受逗号分隔的参数序列，这些参数序列可通过宏 `module_param_array()` 存储在 C 数组中：

```
module_param_array(name, type, nump, perm);
```

参数 `name` 仍然是外部参数以及对应内部变量名，参数 `type` 是数据类型，参数 `perm` 是 `sysfs` 文件系统访问权限，这里新参数是 `nump`，它是一个整型指针，该整型存放数组项数。注意由参数 `name` 指定的数组必须是静态分配的，内核需要在编译时确定数组大小，从而保证不会造成溢

出。该函数用法相当简单，比如：

```
static int fish[MAX_FISH];
static int nr_fish;
module_param_array(fish, int, &nr_fish, 0444);
```

你可以将内部参数数组命名区别于外部参数，这时你需使用宏：

```
module_param_array_named(name, array, type, nump, perm);
```

其中参数和其他宏一致。

最后，你可使用 `MODULE_PARM_DESC()` 描述你的参数：

```
static unsigned short size = 1;
module_param(size, ushort, 0644);
MODULE_PARM_DESC(size, "The size in inches of the fishing pole.");
```

上述所有宏需要包含 `<linux/module.h>` 头文件。

17.2.8 导出符号表

模块被载入后，就会被动态地连接到内核。注意，它与用户空间中的动态链接库类似，只有当被显式导出后的外部函数，才可以被动态库调用。在内核中，导出内核函数需要使用特殊的指令：`EXPORT_SYMBOL()` 和 `EXPORT_SYMBOL_GPL()`。

导出的内核函数可以被模块调用，而未导出的函数模块则无法被调用。模块代码的链接和调用规则相比核心内核镜像中的代码而言，要更加严格。核心代码在内核中可以调用任意非静态接口，因为所有的核心源代码文件被链接成了同一个镜像。当然，被导出的符号表所含的函数必然也要是非静态的。

导出的内核符号表被看做导出的内核接口，甚至称为内核 API。导出符号相当简单，在声明函数后，紧跟上 `EXPORT_SYMBOL()` 指令就搞定了，比如：

```
/*
 * get_pirate_beard_color - 返回当前 pirate 胡须的颜色，
 * @pirate 是一个指向 pirate 结构体的指针；颜色定义在文件 <linux/beard_colors.h> 中
 */
int get_pirate_beard_color(struct pirate *p)
{
    return p->beard.color;
}
EXPORT_SYMBOL(get_pirate_beard_color);
```

假定 `get_pirate_beard_color()` 同时也定义在一个可访问的头文件中，那么现在任何模块都可以访问它。有一些开发者希望自己的接口仅仅对 GPL 兼容的模块可见，内核连接器使用 `MODULE_LICENSE()` 宏可满足这个要求。如果你希望先前的函数仅仅对标记为 GPL 协议的模块可见，那么你就需要用：

```
EXPORT_SYMBOL_GPL(get_pirate_beard_color);
```

如果你的代码被配置为模块，那么你就必须确保当它被编译为模块时，它所用的全部接口都

已被导出，否则就会产生连接错误（而且模块不能成功编译）。

17.3 设备模型

2.6 内核增加了一个引人注目的新特性——统一设备模型（device model）。设备模型提供了一个独立的机制专门来表示设备，并描述其在系统中的拓扑结构，从而使得系统具有以下优点：

- 代码重复最小化。
- 提供诸如引用计数这样的统一机制。
- 可以列举系统中所有的设备，观察它们的状态，并且查看它们连接的总线。
- 可以将系统中的全部设备结构以树的形式完整、有效地展现出来——包括所有的总线和内部连接。
- 可以将设备和其对应的驱动联系起来，反之亦然。
- 可以将设备按照类型加以归类，比如分类为输入设备，而无需理解物理设备的拓扑结构。
- 可以沿设备树的叶子向其根的方向依次遍历，以保证能以正确顺序关闭各设备的电源。

最后一点是实现设备模型的最初动机。若想在内核中实现智能的电源管理，就需要建立表示系统中设备拓扑关系的树结构。当在树上端的设备关闭电源时，内核必须首先关闭该设备节点以下的（处于叶子上的）设备电源。比如内核需要先关闭一个 USB 鼠标，然后才可关闭 USB 控制器；同样内核也必须在关闭 PCI 总线前先关闭 USB 控制器。简而言之，若要准确而又高效地完成上述电源管理目标，内核无疑需要一棵设备树。

17.3.1 kobject

设备模型的核心部分就是 kobject（kernel object），它由 struct kobject 结构体表示，定义于头文件 <linux/kobject.h> 中。kobject 类似于 C# 或 Java 这些面向对象语言中的对象（object）类，提供了诸如引用计数、名称和父指针等字段，可以创建对象的层次结构。

看下面的具体结构：

```
struct kobject {
    const char          *name;
    struct list_head    entry;
    struct kobject      *parent;
    struct kset          *kset;
    struct kobj_type     *ktype;
    struct sysfs_dirent *sd;
    struct kref          kref;
    unsigned int         state_initialized:1;
    unsigned int         state_in_sysfs:1;
    unsigned int         state_add_uevent_sent:1;
    unsigned int         state_remove_uevent_sent:1;
    unsigned int         uevent_suppress:1;
};
```

name 指针指向此 kobject 的名称。